Face Symmetry Measurement: A Quality-Aware, Mirror-Consistent Facial Asymmetry Analysis System

Ankit Yadav (230150) Ashish Porwal (230229) Mahi Mittal (230617)
Pankaj Kumar (230733) Prutha Vinay (230807)
EE604 Image Processing, Course Project, IIT Kanpur

Code: https://github.com/pruthav23/Face-Symmetry-Measurement

Abstract

Public facial-symmetry tools take a single 2D photo, detect landmarks, reflect them about a midline and always return a score, even when the photo cannot support one. We build a system that accepts one face photo and returns a 2D geometric asymmetry measure (global and per region) only when it is trustworthy, together with an accept/caution/reject decision and its reasons. The method has three parts: (i) asymmetry measured with fixed anatomical left-/right pairs and a fitted *symmetry axis, rather than nearest-neighbour matching about an assumed vertical midline; (ii) a* mirror test*, which runs the landmark detector on the photo and on its mirror image and compares the two results, giving a label-free check of landmark reliability and averaging out any left/right detector bias; (iii) a quality gate on face size, sharpness, exposure, head pose and mirror consistency, with thresholds set on a training split. Since no dataset contains true symmetry labels, we evaluate against the same measure computed from human-annotated landmarks (LFPW/300-W, 224 test images) and under 26 controlled perturbations (5,824 perturbed images). The fitted-axis measure agrees far better with this annotation-based reference than a re-implemented public baseline (Spearman ρ 0.947 vs. 0.823); the baseline's KD-tree matching pairs 58.3% of points with the wrong anatomical partner. The mirror test ranks images by landmark error without ground truth (ρ = 0.59), and mirror averaging cuts score drift under perturbation by 29%. The gate reduces silent failures (scores reported with error >1% of the inter-ocular distance) from 8.1% to 3.6%. We also find that on in-the-wild photos the measured 2D asymmetry follows head yaw almost one-to-one (ρ = 0.91), even when computed from expert annotations. This means pose gating is essential, not optional.*

1. Introduction

Bilateral facial symmetry is measured in orthodontics and maxillofacial surgery [2], in outcome assessment after facial plastic surgery [17], and in face analysis research [8]. Dense face-landmark detectors such as MediaPipe Face Landmarker [4, 6] now run in real time on a laptop CPU, which has produced easy-to-use public tools that turn one selfie into a "symmetry score" [5, 11, 20].

Why this is harder than it looks. A 2D photo is a projection. A head turned by a few degrees, a tilted camera, blur, poor lighting or an occluded region all change the measured left–right geometry, and the landmark detector may itself be biased towards a symmetric template. Most existing tools return a number for every input, and the one with input checks [5] does not consider head yaw or pitch. A user cannot tell whether a score of "2%" reflects the face or the photo. Our experiments show that this is a real, large effect: on LFPW the asymmetry measured *from expert annotations* rises from 1.20% to 5.50% of the inter-ocular distance as head yaw grows from $<5^\circ$ to $\geq 20^\circ$ (Fig. 5).

Problem statement. *Input:* a single RGB photo containing a face. *Output:* (a) an accept/caution/reject decision with plain-language reasons; (b) when not rejected, a normalised global asymmetry A and regional values A_r for brows, eyes, nose, mouth and jaw, with a mirror-stability band and a per-region reliability flag; (c) visual overlays and a CSV/JSON export. *Constraints:* 2D images only; a pre-trained detector and explicit geometry, with no model training; CPU real time (46 ms per image); no claims about 3D anatomy, health, identity or attractiveness.

What is new. Landmark detection, mirrored-landmark comparison, regional heatmaps and Streamlit front-ends all already exist in public code [5, 11, 17], and we do not claim them. Our contributions are:

1. **A rotation-invariant measure with correct correspondence.** We pair the 478 MediaPipe vertices using the exactly symmetric canonical face model, map them to the standard 68-point layout with a correspondence fitted on a training split and constrained to be mirror-consistent, and fit the symmetry axis by least squares. We quantify why this matters: the reflected nearest-neighbour (KD-tree) matching used in prior tools [11, 17] picks the wrong anatomical partner for 58.3% of the points.
2. **A mirror test for a symmetry measurement.** Horizontal-flip test-time augmentation is standard in pose and landmark estimation [12, 18], where it is used for accuracy. We use it differently: *flipping the image cannot change a face's asymmetry*, so the disagreement between the two detections is a label-free reliability signal, and their average removes any systematic left/right detector bias from the asymmetry estimate. In the public symmetry tools we audited (Tab. 1) the score is computed from a single pass.
3. **A calibrated quality/pose gate with region validity**, whose thresholds are fixed a priori or set on training data only. It is evaluated as *selective measurement* [3]: how many unreliable scores it prevents and how much coverage it costs.
4. **An evaluation protocol without symmetry labels.** No public dataset has ground-truth symmetry. We compare detector-based measurements with the identical measure computed from expert landmark annotations, and run 26 controlled perturbations. This gives the findings above, plus two that matter in practice: detector-based asymmetry is shrunk towards symmetry (by 12.4% overall, 26.7% at the mouth), and the mirror test does *not* detect an occluded eye.

2. Related Work

Facial landmark localisation. Benchmarks such as LFPW [1] and 300-W [14, 15] (68 points) and WFLW [19] (98 points, with pose, expression, illumination, make-up, occlusion and blur subsets) drove accurate in-the-wild alignment. MediaPipe Face Mesh / Face Landmarker [4, 6, 9] regresses 468 surface vertices (+10 iris points) of a fixed canonical topology and a head-pose transformation matrix in real time on CPU. It returns no per-landmark confidence (its `visibility` and `presence` fields are empty); this gap is what motivated our mirror test.

Quantifying facial asymmetry. Clinical 2D methods compare paired landmarks about a reference midline [2]. Geometric morphometrics avoids choosing a midline by superimposing a configuration with its relabelled reflection, separating symmetric and asymmetric shape components [7, 10]. Our fitted axis follows this idea, restricted to a pure 2D reflection. Liu *et al*. [8] used image-based asymmetry for identification.

	Landm.	Pairing	Axis	Regions	Gate	Eval.
FacialAsym. [11]	MP 478	KD-tree	vertical	7	–	–
Varghaei [17]	MP 468	KD-tree	vertical	–	frontal	private
yousrajnahi [20]	dlib 68	per feature	landm. line	7	–	–
irimid [5]	MP 478	20 fixed	vertical	5	fixed rules*	–
Ours	MP→68	fixed	**fitted**	5	**7 signals**	**LFPW+pert.**

Table 1. **What existing public implementations do** (from their READMEs and code, 5 Nov 2026). "Eval.": quantitative evaluation reported. *resolution, brightness, sharpness, roll, occlusion heuristic; no yaw/pitch. None of the tools checks landmark consistency (our mirror test).

Public symmetry tools. Varghaei *et al*. [17] assess facial-plastic-surgery outcomes with frontal-image filtering, MediaPipe landmarks, KD-tree based mirrored matching and other morphometrics. Their surgical dataset is not public. The *FacialAsymmetry* tool [11] implements that symmetry method in Streamlit, with eye-based alignment, a vertical midline through vertices 33/263, KD-tree matching, seven regions and a heat map. *Facial-symmetry-estimation* [20] makes binary symmetric/asymmetric decisions for individual dlib 68-point features with a fixed tolerance. *irimid* [5] computes anthropometric proportions and a symmetry index from 20 hand-picked MediaPipe vertex pairs about a vertical median midline. It also validates inputs with fixed rules on resolution, brightness, sharpness, in-plane tilt and a span-based occlusion heuristic. Tab. 1 summarises the tools. Quality checks therefore exist, but none of these tools gates on out-of-plane pose (yaw, pitch), checks the landmarks themselves, calibrates its thresholds on data, or reports a quantitative evaluation of its score.

Selective prediction and quality assessment. A model that may abstain is evaluated with risk–coverage curves [3]. We apply this to measurements. Our image-quality signals are classical: variance of the Laplacian for focus [13] and luminance statistics for exposure.

3. Proposed Method

Fig. 1 shows the pipeline. Each step is described below, together with the reason it is there.

3.1. Landmarks and fixed bilateral correspondence

Detection. We use the MediaPipe Face Landmarker (Tasks API, float16 model) in image mode. It returns $L \in \mathbb{R}^{478\times 2}$ in pixels and a 4×4 canonical-to-camera transformation matrix, from which we read yaw, pitch and roll (ZYX Euler angles). *Why:* it runs on CPU in real time,

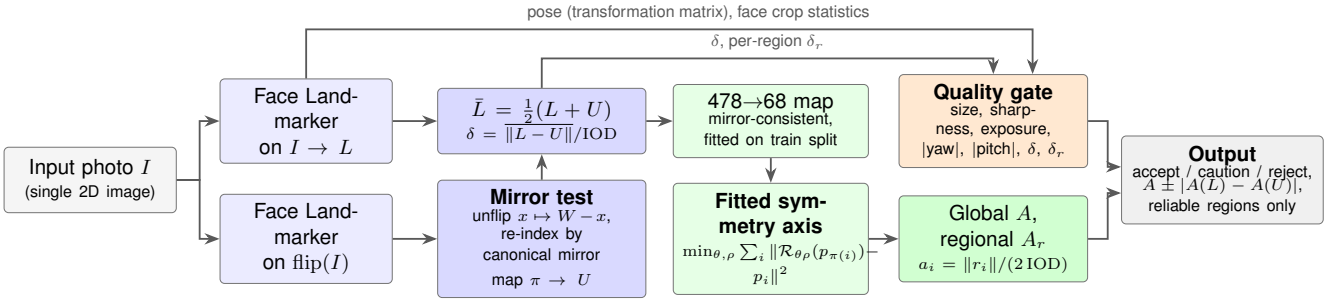


Figure 1. **Overview of the proposed quality-aware measurement (variant M3).** The landmarker is run on the photo and on its mirror image. Because a horizontal flip only swaps anatomical sides, an ideal detector returns the same geometry twice; the disagreement δ is a ground-truth-free check of landmark reliability, and the average $\bar{L}$ cancels any left/right bias of the detector. Asymmetry is measured on the standard 68-point layout with fixed anatomical pairs and a fitted (not assumed) symmetry axis, and a score is only reported when the quality/pose gate does not reject the image.

needs no training, and its mesh has a *fixed topology*, which makes anatomical correspondence possible.

Mirror permutation π. MediaPipe's canonical face model is exactly left–right symmetric. Reflecting it ($x \mapsto -x$) and taking the nearest vertex gives an involution π on the 468 face vertices, with a maximum residual of 0: 220 bilateral pairs and 28 midline vertices (iris points are paired explicitly). A vertex is therefore always compared with its own anatomical twin. *Why not KD-tree matching:* prior tools [11, 17] reflect each left-side vertex and take the *nearest* right-side vertex. In a dense mesh the nearest vertex is often a neighbour of the true twin, so the matching absorbs part of the asymmetry it should measure. On LFPW, 58.3% of KD-tree matches are not the anatomical partner (Fig. 2).

478→68 mapping. We report on the 300-W 68-point layout, whose bilateral pairs are defined by the annotation protocol and for which expert annotations exist. For each iBUG point k and mesh vertex v we compute E_{kv}, the median over the 810 *training* images of $\|L_v - G_k\|/\mathrm{IOD}$, and choose, for each iBUG pair (a, b), the vertex $v^\star = \arg\min_v E_{av} + E_{b\pi(v)}$, setting $a \to v^\star$, $b \to \pi(v^\star)$. Midline points are mapped to midline vertices. The resulting map is *mirror-consistent* by construction ($\pi \circ m = m \circ \pi_{68}$), so the mapping itself cannot create asymmetry. On the test split it reduces the inner-51-point NME from 4.23% (a widely used hand-made map) to 3.95% (median 3.13%), and the jaw NME from 10.11% to 8.13%.

3.2. Asymmetry with a fitted symmetry axis

Let p_i be the 68 points, π their mirror permutation and $\mathcal{R}_{\theta,\rho}(x) = x - 2(n^\top x - \rho)\, n$ the reflection about the line $n^\top x = \rho$, $n = (\cos\theta, \sin\theta)$. We fit the axis on the inner face ($\mathcal{I}$: brows, eyes, nose, mouth; 51 points):

$$(\theta^\star, \rho^\star) = \arg\min_{\theta,\rho} \sum_{i \in \mathcal{I}} \left\| \mathcal{R}_{\theta,\rho}(p_{\pi(i)}) - p_i \right\|^2, \qquad (1)$$

initialised in closed form from pair midpoints and pair directions, then refined with Nelder–Mead. With residuals $r_i = \mathcal{R}_{\theta^\star,\rho^\star}(p_{\pi(i)}) - p_i$,

$$a_i = \frac{\|r_i\|}{2\,\mathrm{IOD}}, \qquad A = \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} a_i, \qquad A_r = \frac{1}{|r|} \sum_{i \in r} a_i, \qquad (2)$$

where IOD is the outer-eye-corner distance (the 300-W normaliser). a_i is the distance of point i from the nearest exactly mirror-symmetric shape, as a fraction of IOD, and we report $100A$ ("% IOD"). *Why:* (i) the axis is estimated, not assumed. A vertical line through two chosen landmarks makes any head roll, or any error in those two landmarks, look like asymmetry. Equation (2) is invariant to rotation, translation and scale of the image (unit-tested). (ii) Unlike Procrustes superimposition, a pure reflection cannot hide asymmetry in a glide component. (iii) The 17-point jaw contour is a pose-dependent silhouette, so it is reported as a region but excluded from A. For readability the app also shows $S = 100/(1 + \alpha A)$ with α=0.467 (the training median, 2.14% IOD, maps to 50). S is only a monotone re-expression of A, not a validated scale.

3.3. The mirror test

We run the detector on I and on its horizontal flip I^f. A point at x' in I^f lies at $W - x'$ in I (MediaPipe uses continuous coordinates with pixel centres at $i + \frac{1}{2}$), and flipping swaps anatomical sides, so the vertex reported as j in I^f is vertex $\pi(j)$ of I:

$$U_{\pi(j)} = \left(W - L^f_{j,x},\ L^f_{j,y}\right). \qquad (3)$$

For an ideal detector $U = L$. We define the *mirror inconsistency* $\delta = \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \|L_i - U_i\|/\mathrm{IOD}$ (and per region, δ_r), and measure asymmetry on the average $\bar{L} = \frac{1}{2}(L + U)$. *Why:* the true asymmetry of a face does not change when the image is flipped, but a detector's errors do not have to

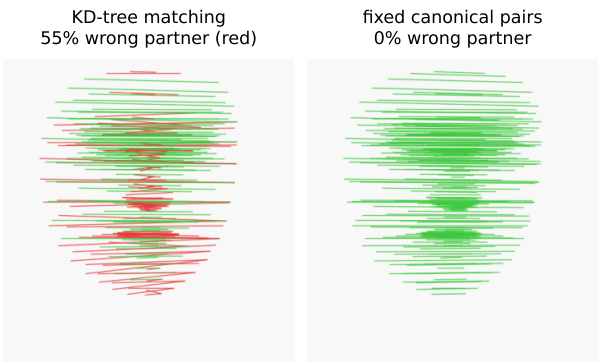


Figure 2. **Correspondence matters.** Lines join each left-side vertex to its partner for one test face (geometry only; see Sec. 5). Left: baseline KD-tree matching after reflection; red lines are matches that are not the anatomical twin. Right: fixed pairs from the symmetric canonical model.

flip with it. The disagreement δ therefore estimates landmark unreliability *without ground truth*. MediaPipe exposes no per-landmark confidence, so δ fills that gap. Averaging cancels any systematic left/right bias of the detector, and $|A(L) - A(U)|$ is shown to the user as a stability band. The cost is one extra detector pass (46 ms per image in total, including the baseline's re-detection, on a 2-core 2.1 GHz Xeon).

3.4. Quality and pose gate

The gate reads seven signals that the system can actually observe (Tab. 2): **face size** (IOD in pixels); **sharpness**, $\log_{10}$ of the variance of the Laplacian [13] on the face crop resized to 256 px width, so that it does not depend on face size; **exposure**, the mean luma and the fraction of clipped pixels in the face crop; **|yaw|, |pitch|** from the transformation matrix; and the **mirror inconsistency** δ. In-plane roll is *not* gated, because Eq. (1) is invariant to it. The worst signal decides accept / caution / reject, and every decision is shown with a plain-language reason. A rejected image never shows a score unless the user explicitly asks for it, and it is then labelled unreliable. In addition, a region whose δ_r exceeds a training percentile is marked *unreliable* and greyed out.

Threshold policy (fixed before looking at test results): pose and face-size limits follow from the measurement's assumptions (yaw 10°/20°, pitch 15°/25°, IOD 60/30 px). Image-statistic limits are percentiles of the *training* split: 2.5th percentile of sharpness and of brightness for caution, and 90th/99th percentiles of δ. The test split is used only for evaluation.

3.5. Compared variants

All variants use the *same* detections of each image: **M0** (baseline) is our re-implementation of [11]: similarity alignment of vertices 33/263 to fixed positions, re-detection, a vertical midline through 33/263, KD-tree reflected matching on all vertices, and the mean distance divided by face width. That repository has no licence, so we re-implemented the method from its description and code rather than copying it. Note that 33/263 are the *outer* eye corners, although the code comments call them inner corners. **M1**: fixed 68-point pairs + fitted axis (Secs. 3.1 and 3.2). **M2**: M1 + gate without δ. **M3** (full system): measurement on $\bar{L}$, gate with δ, region validity.

Signal	caution	reject	source
face size (IOD)	< 60 px	< 30 px	a priori
sharpness ($\log_{10}$ var. Lap.)	< 1.4	< 0.756	train pct.
mean luma (low)	< 59.3	< 35	train pct. / a priori
mean luma (high)	> 164	> 225	train pct. / a priori
clipped pixels	> 0.364	> 0.6	train pct. / a priori
|yaw|	$> 10°$	$> 20°$	a priori
|pitch|	$> 15°$	$> 25°$	a priori
mirror inconsistency δ	> 0.0224	> 0.0489	train pct.
per-region δ_r (validity)	unreliable if > 0.0444		train pct.

Table 2. **Gate thresholds** (`configs/gate_thresholds.json`). "train pct.": percentile of the 810 LFPW training faces.

Split	Images	Faces detected	Used for
LFPW train	811	810	478→68 map, gate calibration
LFPW test	224	224	all evaluation (clean)
Perturbed test	5,824	5,730	robustness (26 settings × 224)

Table 3. **Data.** 68 manual landmarks per face; research-use licence (300-W).

4. Experiments

4.1. Setup

Data. We use LFPW [1] with the 300-W 68-point re-annotation [14, 15], obtained from the Menpo mirror [16] (Tab. 3). Following the 300-W protocol, which supplies a face box, each image is cropped around the annotated face (landmark extent + 60%); many LFPW photos contain several people. The **train** split is used *only* to fit the 478→68 map and to calibrate the gate. Every number below is computed on the **test** split. WFLW [19] was our first choice for its attribute subsets, but its Google-Drive-hosted images were unreachable from our compute environment. The code reads any 68-point `.pts` dataset.

Reference without symmetry labels. No public dataset has ground-truth symmetry ratings. We therefore compute the *same* measure (Eq. (2)) from the expert 68-point annotations, A_{ref}, and evaluate how well each detector-based measure agrees with it (Spearman ρ, Pearson r, MAE). A_{ref} is not "true" asymmetry: it contains annotator noise and the same 2D/pose limitations (Sec. 5). It is the best

Measure (n=224)	ρ	r	MAE
M0 baseline [11] (KD-tree, vert. midline)	0.823	0.762	–
M0 w/o alignment	0.614	0.630	–
M0 + fixed pairs (no KD-tree)	0.931	0.903	–
M1: 68-pt pairs + fitted axis	0.938	0.951	0.45
M3: M1 + mirror averaging	0.947	0.957	0.45

Table 4. **Agreement with** A_{ref} on the clean LFPW test set (E1/E2): Spearman ρ, Pearson r, MAE in % IOD. All rows use identical detections.

available reference for *the measurement* we want to automate.

Perturbations (deterministic, applied to all 224 test faces): brightness $\times\{0.25, 0.4, 0.6, 1.5, 2, 2.5\}$; Gaussian blur $\sigma = s \cdot \mathrm{IOD}/100$, $s \in \{1, 2, 4, 6, 8\}$; a grey box over one eye, the nose or the mouth (annotation box +35%); in-plane rotation $\pm\{5, 10, 20, 30\}^\circ$ on an enlarged canvas; and downscaling $\times\{0.5, 0.3, 0.2, 0.12\}$. The face is unchanged, so A_{ref} is unchanged too.

Metrics. *Drift*: median |score(perturbed)−score(clean)| divided by the clean between-subject SD of that variant. This is scale-free, so M0 (face-width units) and M1/M3 (IOD units) can be compared. *Reported*: fraction of images not rejected. *Silent failure*: fraction of images whose score is *reported* but has $|A - A_{ref}| > 1\%$ IOD (about twice the clean MAE, fixed a priori). This is the failure a gate must prevent. For M0, MAE and silent failure are undefined because its units differ from those of A_{ref}. **Implementation:** Python 3.13, MediaPipe 0.10.32, OpenCV 5.0, NumPy/SciPy, Streamlit 1.66; CPU only. All tables and figures are generated by scripts from the saved CSVs.

4.2. E1–E3: agreement with the annotation-based reference

Tab. 4 and Fig. 4 answer the first question: does the system measure what we intend? The re-implemented baseline reaches ρ=0.823, or 0.614 without its alignment step. Replacing only its KD-tree matching with the fixed canonical pairs, keeping the same detections, mesh and midline, raises ρ to 0.931. *Correspondence is the dominant error source in the baseline.* The 68-point measure with the fitted axis reaches 0.938 (MAE 0.45% IOD), and mirror averaging reaches 0.947.

Regional analysis (E3, Tab. 5). Regional agreement is lower than global agreement (ρ 0.84–0.98) because each region has fewer points. A systematic effect appears: detector-based asymmetry is *smaller* than annotation-based asymmetry, by 12.4% globally and 26.7% at the mouth (Wilcoxon p <0.01 except for the brows, p=0.08). MediaPipe regresses a mesh with a strong shape prior, and this pulls atypical (asymmetric) shapes towards the symmetric template. Part of the gap may also be annotator noise, which inflates A_{ref}. Users should read detector-based regional values as conservative.

	Spearman ρ		MAE (% IOD)		mean A (% IOD)	
Region	M1	M3	M1	M3	ref	M3
Brows	0.840	0.848	0.76	0.76	2.85	2.80†
Eyes	0.882	0.895	0.59	0.56	2.25	2.08
Nose	0.912	0.914	0.87	0.86	3.90	3.53
Mouth	0.876	0.889	0.54	0.56	1.86	1.36
Jaw	0.974	0.977	1.61	1.60	11.73	10.44
Global (inner 51)	–	–	–	–	2.51	2.19

Table 5. **Regional agreement and shrinkage** (E3). Right block: mean asymmetry from annotations vs. M3. †difference not significant (Wilcoxon $p \geq 0.01$).

|yaw| (deg)	n	A_{ref}	M1	|err| M3	NME
0-5	77	1.20	0.80	0.47	3.03
5-10	67	2.14	1.86	0.40	5.51
10-20	60	3.60	3.41	0.42	3.32
≥20	20	5.50	5.42	0.57	4.13

Table 6. **Pose strata** (E8, test set; values in % IOD). Each image is observed once, so the strata contain *different* people, not the same face at several poses.

4.3. E8: head pose confounds 2D asymmetry

Fig. 5 and Tab. 6 show the strongest effect we observed. Even computed from *expert annotations*, 2D asymmetry follows |yaw| almost monotonically (ρ=0.91), from 1.20% IOD below 5° to 5.50% above 20°. A turned head foreshortens one half of the face, and no 2D reflection can undo that. This is a property of the *quantity*, not of the detector. The detector's error relative to A_{ref} barely changes with yaw (Tab. 6), which is why a gate trained to predict detector error would miss it. Restricting to gate-accepted images lowers the correlation to ρ=0.75, but it does not remove it. LFPW is an in-the-wild set whose test faces mostly have |yaw| >5°. For a frontal capture the remaining pose effect is small, but a "symmetry score" computed from an arbitrary selfie mostly measures head pose.

4.4. The mirror test detects bad landmarks without labels

Across test images, δ correlates with the true landmark error (inner-51 NME, ρ=0.59). As a ranking signal for discarding images (Fig. 6, left), it lowers the area under the NME risk–coverage curve from 3.37 (random) to 2.99 (oracle: 2.75), closing 61% of the gap to the oracle. The image-quality gate risk alone reaches 3.29. δ predicts the *asym-*

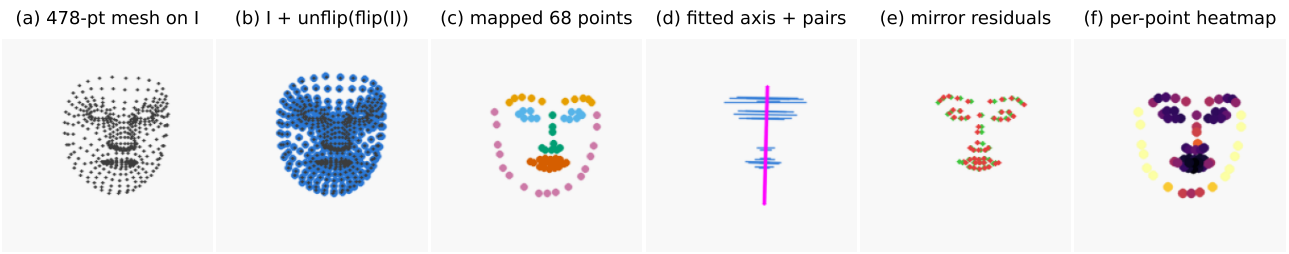


Figure 3. **Intermediate outputs for one test face**, drawn as geometry only. (a) mesh on I; (b) mesh on I (black) and unflipped mesh from flip(I) (blue), almost coincident here (δ small); (c) mapped 68 points coloured by region; (d) bilateral pairs and the fitted axis (magenta); (e) each point (green) and the reflection of its partner (red); (f) per-point a_i (bright = larger).

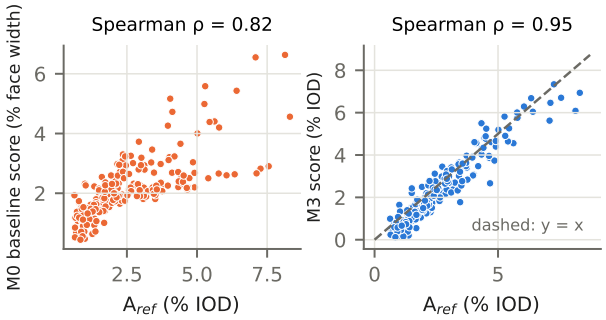


Figure 4. **Detector-based score vs. annotation-based reference** (224 test faces). Left: baseline M0. Right: proposed M3. Points below $y = x$ show that the detector under-estimates asymmetry.

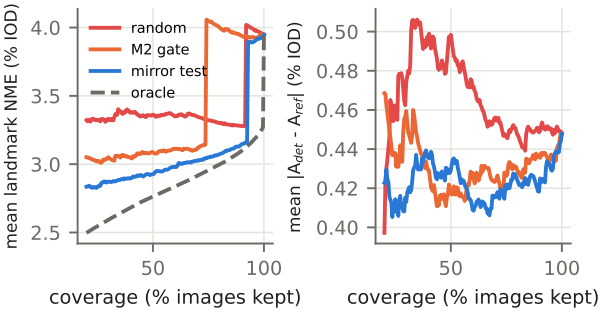


Figure 6. **Risk–coverage on the clean test set.** Images are kept in order of increasing risk; the y-axis is the mean error of the kept set (lower is better). Left: landmark NME. Right: asymmetry error $|A_{det} - A_{ref}|$.

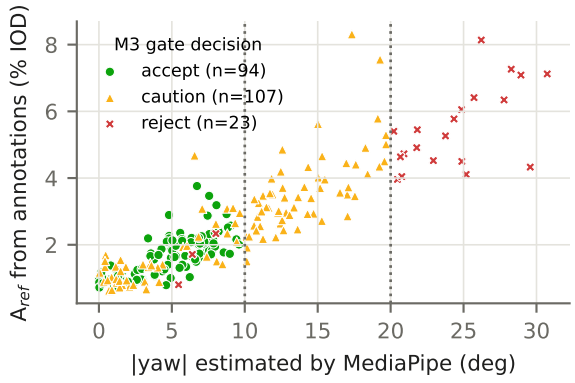


Figure 5. **Asymmetry from expert annotations vs. estimated |yaw|** (E8), coloured by the M3 gate decision. Dotted lines: caution (10°) and reject (20°) limits.

metry error much less well (ρ=0.03; AURC 0.418 vs. 0.442 random; Fig. 6, right). On clean photos, most landmark errors are shared by the two sides of the face and cancel in A. The gate's value for asymmetry shows up under degradation (Sec. 4.5).

4.5. E4–E7, E9: robustness and ablation

Fig. 7 and Tab. 7 summarise 5,824 perturbed measurements. MediaPipe is remarkably robust: it still finds a face in $\geq$79.0% of the images in every setting (lowest: blur s=8). The danger is therefore not a missing result but a *plausible-looking wrong* one. Without a gate (M1), 8.1% of all perturbed measurements are silent failures, rising to 14.4% under blur. The gate (M2/M3) lowers this to 3.4%/3.6% (blur: 2.3%; downscaling: 9.2%→1.9%). The cost is coverage: 63.2% of perturbed and 89.7% of clean images still receive a score. On clean images silent failures fall from 5.8% to 4.0%. **Mirror averaging** lowers the median drift from 0.076 to 0.054 SD (29% less, pooled) and helps under *every* perturbation type. **In-plane rotation** (E7): a vertical-midline measure without alignment drifts by up to 3.67 SD at 30°. The baseline's align-and-re-detect step fixes this, at the cost of a second detection. The fitted axis achieves the same stability (M3 $\leq$ 0.052 SD) with no re-detection.

Gate behaviour on clean photos. On the clean test set M3 accepts 42.0%, cautions 47.8% and rejects 10.3% of the faces. The mean NME of accepted faces is 3.03% vs. 4.71% for caution. Tab. 8 lists the triggers; pose dominates,

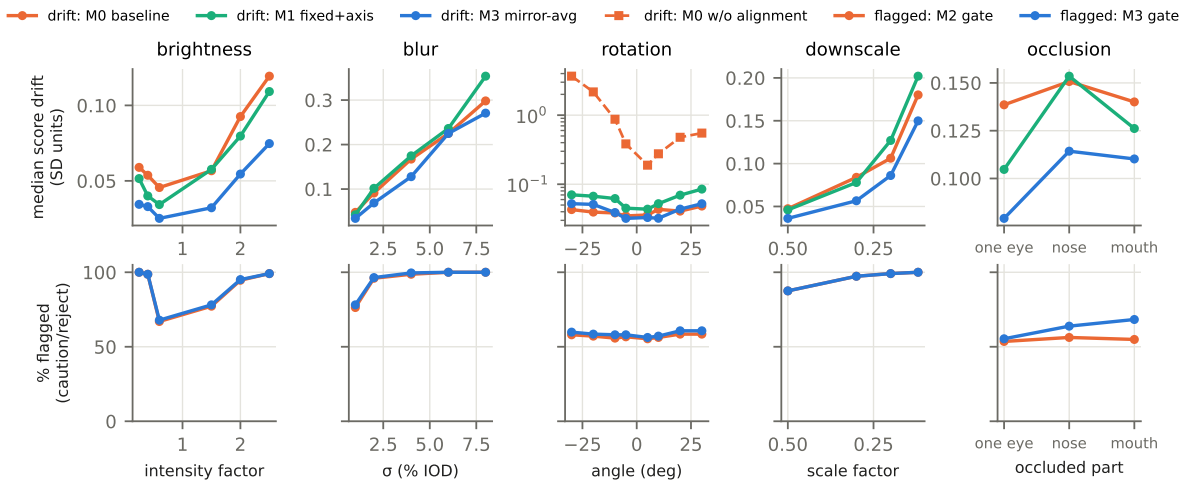


Figure 7. **Controlled perturbations** (E4–E7; 224 test faces per point). Top: median score drift in units of the clean between-subject SD (rotation panel in log scale; dashed: vertical-midline baseline without alignment). Bottom: share of images the gate flags as caution or reject.

Setting	Var.	n	Reported (%)	Accepted (%)	ρ vs ref	Silent fail. (%) ↓	Drift (SD) ↓
Clean test	M0	224	100.0	100.0	0.823	–	–
	M1		100.0	100.0	0.938	5.8	–
	M2		90.2	43.3	0.921	4.9	–
	M3		89.7	42.0	0.932	4.0	–
Perturbed, pooled	M0	5824	98.4	98.4	0.814	–	0.070
	M1		98.4	98.4	0.921	8.1	0.076
	M2		64.0	22.6	0.918	3.4	=
	M3		63.2	21.0	0.927	3.6	0.054
blur	M0	1120	94.0	94.0	0.787	–	0.131
	M1		94.0	94.0	0.870	14.4	0.130
	M2		37.0	5.8	0.903	2.7	=
	M3		35.6	5.2	0.919	2.3	0.099
downscale	M0	896	98.7	98.7	0.821	–	0.087
	M1		98.7	98.7	0.909	9.2	0.097
	M2		33.5	4.0	0.919	1.8	=
	M3		33.4	4.0	0.927	1.9	0.066
brightness	M0	1344	99.5	99.5	0.818	–	0.065
	M1		99.5	99.5	0.936	5.8	0.056
	M2		62.8	10.6	0.927	2.8	=
	M3		62.4	10.2	0.940	2.7	0.037
occlusion	M0	672	98.8	98.8	0.804	–	0.142
	M1		98.8	98.8	0.919	8.8	0.124
	M2		87.9	45.1	0.900	6.2	=
	M3		85.1	37.5	0.911	6.0	0.098
rotation	M0	1792	100.0	100.0	0.825	–	0.039
	M1		100.0	100.0	0.944	5.2	0.059
	M2		88.1	43.0	0.924	4.0	=
	M3		87.7	41.4	0.928	4.9	0.040

Table 7. **Ablation** (E9). Reported = not rejected; Accepted = no caution. Silent fail.: reported *and* $|A - A_{ref}| > 1\%$ IOD. Drift: median, in clean-SD units ("=": same score as M1).

consistent with E8. Fig. 8 shows one face per decision.

Region validity under occlusion: a negative result. When the nose or the mouth is occluded, δ_r marks that region unreliable in 17.5% and 18.4% of the faces, against 8.5% and 2.2% for the other regions. When an *eye* is occluded it does so in only 0.9% of faces. MediaPipe fills in the hidden eye from its shape prior, and does so consistently in both the original and the mirror image. The mirror test detects *inconsistent* landmarks, not *confidently invented* ones. Occlusion needs an appearance-based check (future work).

Signal	caution	reject
|yaw|	60	20
|pitch|	41	2
mirror inconsistency δ	12	1
face size (IOD)	16	0
over-exposure	10	0
sharpness	7	0
several faces	6	0
clipped pixels	4	0
under-exposure	3	0

Table 8. **Which signal triggered the gate** on the 224 clean test faces (one image can trigger several).

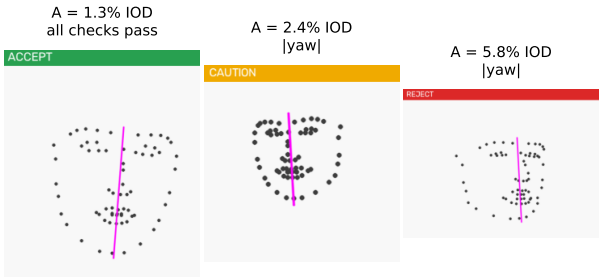


Figure 8. **Gate decisions** for three test faces (the median-|yaw| face of each class), drawn as landmarks + fitted axis.

5. Discussion and Limitations

What worked. Two changes give most of the gain over the public baseline: using the anatomical correspondence that the mesh already provides, and estimating the symmetry axis instead of assuming it. The mirror test is cheap, needs no labels, and gives the system a landmark-reliability signal that MediaPipe does not expose. The gate turns many silent failures into explicit refusals.

What did not. The mirror test cannot detect an occluded eye that the detector fills in consistently. On clean photos the gate does not predict small asymmetry errors better than chance, because those errors are dominated by symmetric landmark noise and by the detector's shrinkage towards symmetry.

Limitations.

- **2D only.** A photo is a projection. Yaw and pitch change the measured value (E8), and the gate can only limit, not remove, that effect. Camera perspective at selfie distance is not modelled.
- **Reference, not ground truth.** A_{ref} uses expert landmarks, which contain annotator noise, and is subject to the same projection effects. We have no human symmetry ratings, and a validated perceptual or clinical scale was out of scope.
- **Dataset.** LFPW is small (224 test faces), in the wild and mostly non-frontal, and each identity appears once, so the pose strata are not paired. MediaPipe's training data is not public and could overlap with LFPW, so its NME here is not an independent benchmark claim. We use it only to compare mappings and gate signals.
- **Thresholds** are percentiles of one training distribution. A different camera or population would need re-calibration (`calibrate_gate.py`).
- **Expression** is not controlled. A smile is asymmetric by nature, so the app asks for a neutral expression.

Ethics and privacy. The output is a geometric indicator. It says nothing about health, identity or attractiveness, and the app states this. Dataset images are used under their research licence and are neither redistributed in the repository nor reproduced in this report. Figures draw landmark geometry on a blank canvas. The app processes uploads in memory and stores nothing.

6. Conclusion and Future Work

We built and evaluated a quality-aware facial asymmetry system that reports a measurement only when the image supports it. On LFPW it tracks an annotation-based reference much more closely than a re-implemented public baseline (ρ 0.947 vs. 0.823). Its mirror test flags unreliable landmarks without labels, and its gate cuts silent failures under degradation from 8.1% to 3.6%. The study also gives a practical warning: in uncontrolled photos, head yaw explains most of the measured 2D asymmetry. Future work: frontalisation with the detector's 3D landmarks, evaluated on a dataset with ground-truth pose (*e.g.* AFLW2000-3D [21]); appearance-based occlusion detection; WFLW's attribute subsets; and a small consented, multi-pose capture of the same people with human symmetry ratings.

References

[1] Peter N. Belhumeur, David W. Jacobs, David J. Kriegman, and Neeraj Kumar. Localizing parts of faces using a consensus of exemplars. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 545–552, 2011. 2, 4

[2] Nina F. Berlin, Philipp Berssenbrügge, Christoph Runte, Kai Wermker, Susanne Jung, Johannes Kleinheinz, and Dieter Dirksen. Quantification of facial asymmetry by 2D analysis – a comparison of recent approaches. *Journal of Cranio-Maxillofacial Surgery*, 42(3):265–271, 2014. 1, 2

[3] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. 2

[4] Google AI Edge. MediaPipe face landmarker for Python. `https://developers.google.com/edge/mediapipe/solutions/vision/face_landmarker/python`. Accessed 5 Nov 2026. 1, 2

[5] irimidlab. irimid: facial anthropometry and geometry toolkit. `https://github.com/irimidlab/irimid`. Accessed 5 Nov 2026. 1, 2

[6] Yury Kartynnik, Artsiom Ablavatski, Ivan Grishchenko, and Matthias Grundmann. Real-time facial surface geometry from monocular video on mobile GPUs. In *CVPR Workshop on Computer Vision for Augmented and Virtual Reality*, 2019. arXiv:1907.06724. 1, 2

[7] Christian Peter Klingenberg, Marta Barluenga, and Axel Meyer. Shape analysis of symmetric structures: Quantifying variation among individuals and asymmetry. *Evolution*, 56(10):1909–1920, 2002. 2

[8] Yanxi Liu, Karen L. Schmidt, Jeffrey F. Cohn, and Sinjini Mitra. Facial asymmetry quantification for expression invariant human identification. *Computer Vision and Image Understanding*, 91(1–2):138–159, 2003. 1, 2

[9] Camillo Lugaresi, Jiuqiang Tang, Hadon Nash, Chris McClanahan, Esha Uboweja, Michael Hays, Fan Zhang, Chuo-Ling Chang, Ming Guang Yong, Juhyun Lee, Wan-Teh Chang, Wei Hua, Manfred Georg, and Matthias Grundmann. MediaPipe: A framework for building perception pipelines. *arXiv preprint arXiv:1906.08172*, 2019. 2

[10] Kanti V. Mardia, Fred L. Bookstein, and Ian J. Moreton. Statistical assessment of bilateral symmetry of shapes. *Biometrika*, 87(2):285–300, 2000. 2

[11] nashokan. FacialAsymmetry: Automated facial asymmetry analysis tool. https://github.com/nashokan/FacialAsymmetry. Commit 46c0d98, accessed 5 Nov 2026; no licence file, used as a reference only. 1, 2, 3, 4, 5

[12] Alejandro Newell, Kaiyu Yang, and Jia Deng. Stacked hourglass networks for human pose estimation. In *European Conference on Computer Vision (ECCV)*, pages 483–499, 2016. 2

[13] Said Pertuz, Domenec Puig, and Miguel Angel Garcia. Analysis of focus measure operators for shape-from-focus. *Pattern Recognition*, 46(5):1415–1432, 2013. 2, 4

[14] Christos Sagonas, Epameinondas Antonakos, Georgios Tzimiropoulos, Stefanos Zafeiriou, and Maja Pantic. 300 faces in-the-wild challenge: Database and results. *Image and Vision Computing*, 47:3–18, 2016. 2, 4

[15] Christos Sagonas, Georgios Tzimiropoulos, Stefanos Zafeiriou, and Maja Pantic. 300 faces in-the-wild challenge: The first facial landmark localization challenge. In *Proceedings of the IEEE International Conference on Computer Vision Workshops (ICCVW)*, pages 397–403, 2013. 2, 4

[16] The Menpo Project. LFPW train/test images with 300-w annotations (menpo mirror). https://github.com/menpo/lfpw-train, https://github.com/menpo/lfpw-test. Accessed 5 Nov 2026. 4

[17] Pegah Varghaei et al. Automated assessment of aesthetic outcomes in facial plastic surgery. In *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, 2025. arXiv:2508.13363. Code: https://github.com/varghaei/face-aesthetics-analysis. 1, 2, 3

[18] Jingdong Wang, Ke Sun, Tianheng Cheng, Borui Jiang, Chaorui Deng, Yang Zhao, Dong Liu, Yadong Mu, Mingkui Tan, Xinggang Wang, Wenyu Liu, and Bin Xiao. Deep high-resolution representation learning for visual recognition. volume 43, pages 3349–3364, 2021. 2

[19] Wayne Wu, Chen Qian, Shuo Yang, Quan Wang, Yici Cai, and Qiang Zhou. Look at boundary: A boundary-aware face alignment algorithm. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2129–2138, 2018. 2, 4

[20] yousrajnahi. Facial-symmetry-estimation. https://github.com/yousrajnahi/Facial-symmetry-estimation. Accessed 5 Nov 2026. 1, 2

[21] Xiangyu Zhu, Zhen Lei, Xiaoming Liu, Hailin Shi, and Stan Z. Li. Face alignment across large poses: A 3D solution. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 146–155, 2016. 8

A. Team Contributions

- **Prutha Vinay (230807)**: project coordination; symmetry geometry engine (fitted axis, mirror test, `geometry.py`, `pipeline.py`); integration and repository maintenance.
- **Ankit Yadav (230150)**: dataset pipeline (LFPW/300-W loading, face crops), the mirror-consistent 478→68 mapping fit and the landmark (NME) evaluation.
- **Ashish Porwal (230229)**: quality/pose gate design and calibration (`quality.py`, `calibrate_gate.py`); perturbation experiments E4–E7.
- **Mahi Mittal (230617)**: Streamlit application and user interface, visualisation module and paper figures.
- **Pankaj Kumar (230733)**: literature and repository audit, re-implementation of the baseline (M0), ablation analysis and report/presentation writing.

All members reviewed the results and the report.

B. Code, Data and Resource Links

Project repository: https://github.com/pruthav23/Face-Symmetry-Measurement
Run: `pip install -r requirements.txt`; `streamlit run app.py`. Reproduce all results: see `README.md` (`experiments/` scripts in order).
Datasets. LFPW/300-W: https://ibug.doc.ic.ac.uk/resources/300-W/; annotation scheme: https://ibug.doc.ic.ac.uk/resources/facial-point-annotations/; mirror used: https://github.com/menpo/lfpw-train, https://github.com/menpo/lfpw-test; WFLW (considered): https://github.com/wywu/LAB; CelebA (considered): https://mmlab.ie.cuhk.edu.hk/projects/CelebA.html.
Repositories and tools. MediaPipe Face Landmarker: https://developers.google.com/edge/mediapipe/solutions/vision/face_landmarker/python; canonical face model: https://github.com/google-ai-edge/mediapipe/tree/master/mediapipe/modules/face_geometry/data; baseline method: https://github.com/nashokan/FacialAsymmetry; https://github.com/varghaei/face-aesthetics-analysis; https://github.com/yousrajnahi/Facial-symmetry-estimation; https://github.com/irimidlab/irimid; CVPR author kit: https://github.com/cvpr-org/author-kit. No code was copied from the audited repositories (none has a licence file); the baseline was re-implemented from its documented method.

C. Complete Source Code

The complete source code is listed below and is also available in the repository. Unit tests: `tests/test_geometry.py` (15 tests, all passing).

src/fsm/mesh.py

```
"""
mesh.py -- landmark topologies, bilateral correspondences and anatomical regions.

Two landmark layouts are used in this project:

1. The 478-point MediaPipe Face Landmarker mesh (468 face vertices + 10 iris points).
   Its bilateral correspondence is NOT guessed per image: it is derived once from
   Google's canonical face model (configs/canonical_face_model.obj), which is exactly
   left-right symmetric. Vertex i is paired with the vertex whose canonical position
   is the mirror image (-x, y, z) of vertex i.

2. The 68-point iBUG / 300W layout used by the LFPW human annotations. Its bilateral
   pairs are fixed by the published annotation scheme (e.g. point 37 <-> 46 for the
   outer eye corners, 1-indexed). All reported asymmetry measures are computed on this
   layout so that detector-based measurements can be compared with measurements from
   human annotations of the same points.

The MediaPipe->68 correspondence (which mesh vertex plays the role of each iBUG point)
is *fitted on the LFPW training split* by experiments/fit_mp68_mapping.py and stored in
configs/mp68_mapping.json. It is constrained to be mirror-consistent (see that script).
"""
from __future__ import annotations

import json
import os
from functools import lru_cache

import numpy as np
from scipy.spatial import cKDTree

ROOT = os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
CONFIG_DIR = os.path.join(ROOT, "configs")

# ---------------------------------------------------------------------------
# 68-point iBUG layout (0-indexed)
# ---------------------------------------------------------------------------
# Bilateral pairs (left, right) following the 300W markup. Points not listed are on the
# facial midline and map to themselves under reflection.
IBUG68_PAIRS = (
    # jaw line 0..16 (8 = chin is central)
    [(i, 16 - i) for i in range(8)]
    # eyebrows 17..21 <-> 26..22
    + [(17 + i, 26 - i) for i in range(5)]
    # lower nose 31,32 <-> 35,34 (33 = subnasale is central; 27..30 bridge is central)
    + [(31, 35), (32, 34)]
    # eyes: 36..41 (one eye) <-> 45..42,47,46 (other eye)
    + [(36, 45), (37, 44), (38, 43), (39, 42), (40, 47), (41, 46)]
    # outer lips: 48<->54, 49<->53, 50<->52, 59<->55, 58<->56 (51, 57 central)
    + [(48, 54), (49, 53), (50, 52), (59, 55), (58, 56)]
    # inner lips: 60<->64, 61<->63, 67<->65 (62, 66 central)
    + [(60, 64), (61, 63), (67, 65)]
)
IBUG68_CENTRAL = [8, 27, 28, 29, 30, 33, 51, 57, 62, 66]

# Anatomical regions of the 68-point layout used for regional reporting.
IBUG68_REGIONS = {
    "Brows": list(range(17, 27)),
    "Eyes": list(range(36, 48)),
    "Nose": list(range(27, 36)),
    "Mouth": list(range(48, 68)),
    "Jaw": list(range(0, 17)),
}
# Inner-face points (everything except the jaw contour). The jaw contour of 300W is a
# "silhouette" whose exact position along the cheek is ill-defined, and MediaPipe's
# face oval does not correspond to it point-for-point, so headline numbers use the
# inner 51 points and the jaw is reported separately.
IBUG68_INNER = list(range(17, 68))

# Outer eye corners: inter-ocular distance (IOD) normaliser, as in the 300W NME metric.
IBUG68_OUTER_EYE = (36, 45)


def ibug68_permutation() -> np.ndarray:
    """Return perm[i] = index of the mirror partner of point i (central points -> self)."""
    perm = np.arange(68)
    for a, b in IBUG68_PAIRS:
        perm[a], perm[b] = b, a
    return perm


# ---------------------------------------------------------------------------
# 478-point MediaPipe mesh
# ---------------------------------------------------------------------------
MP_N = 478
MP_IRIS = list(range(468, 478))  # 468 = one iris centre, 469-472 ring; 473 = other centre, 474-477 ring


@lru_cache(maxsize=1)
```

```
def canonical_vertices() -> np.ndarray:
    """Load the 468 canonical face-model vertices (x, y, z) shipped with MediaPipe."""
    path = os.path.join(CONFIG_DIR, "canonical_face_model.obj")
    verts = [list(map(float, ln.split()[1:4])) for ln in open(path) if ln.startswith("v ")]
    return np.asarray(verts)


@lru_cache(maxsize=1)
def mp_permutation() -> np.ndarray:
    """
    Mirror permutation of the 478-point mesh.

    For the 468 face vertices: reflect the canonical model (x -> -x) and find the vertex
    at the reflected position (the canonical model is exactly symmetric, max error 0).
    Iris centres 468 <-> 473 are paired; iris ring points are paired by their order
    (469..472 <-> 474..477), which is verified empirically by the flip test in tests/.
    """
    V = canonical_vertices()
    M = V.copy()
    M[:, 0] *= -1.0
    _, idx = cKDTree(V).query(M)
    perm = np.arange(MP_N)
    perm[:468] = idx
    # iris: centre<->centre; ring points paired so that horizontal neighbours swap
    iris_a = [468, 469, 470, 471, 472]
    iris_b = [473, 476, 475, 474, 477]  # see tests/test_mesh.py::test_iris_flip_pairing
    for a, b in zip(iris_a, iris_b):
        perm[a], perm[b] = b, a
    return perm


def mp_bilateral_pairs() -> np.ndarray:
    """(K, 2) array of mesh-vertex pairs (i, mirror(i)) with i < mirror(i)."""
    perm = mp_permutation()
    return np.array([(i, j) for i, j in enumerate(perm) if i < j], dtype=int)


@lru_cache(maxsize=1)
def mp68_mapping() -> np.ndarray:
    """
    Load the fitted MediaPipe-vertex index for each of the 68 iBUG points.
    Falls back to a commonly used hand-made mapping if the fitted file is absent.
    """
    path = os.path.join(CONFIG_DIR, "mp68_mapping.json")
    if os.path.exists(path):
        with open(path) as f:
            return np.asarray(json.load(f)["mp_index_for_ibug68"], dtype=int)
    return np.asarray(HANDMADE_MP68, dtype=int)


# A widely circulated hand-made MediaPipe->68 correspondence (used only as a starting
# point / fallback; the fitted mapping replaces it).
HANDMADE_MP68 = [
    162, 234, 93, 58, 172, 136, 149, 148, 152, 377, 378, 365, 397, 288, 323, 454, 389,
    71, 63, 105, 66, 107, 336, 296, 334, 293, 301,
    168, 197, 5, 4, 75, 97, 2, 326, 305,
    33, 160, 158, 133, 153, 144, 362, 385, 387, 263, 373, 380,
    61, 39, 37, 0, 267, 269, 291, 405, 314, 17, 84, 181,
    78, 82, 13, 312, 308, 317, 14, 87,
]


def mp_to_68(lm478: np.ndarray) -> np.ndarray:
    """Select the 68 mesh vertices that play the role of the iBUG points."""
    return lm478[mp68_mapping()]
```

src/fsm/geometry.py

```
"""
geometry.py -- the symmetry mathematics. No image processing and no UI in this file,
so everything here is unit-tested directly (tests/test_geometry.py).

Notation
--------
P      : (n, 2) landmark coordinates in pixels (x right, y down).
perm   : mirror permutation, perm[i] is the bilateral partner of point i
         (perm[i] == i for midline points).
axis   : the symmetry line  {x : n . x = rho}, with unit normal n = (cos t, sin t).
refl(x) = x - 2 (n . x - rho) n   is the reflection about that line.

Proposed measure (used by variants M1-M3)
-----------------------------------------
Instead of assuming a vertical midline through two chosen landmarks, we estimate the
symmetry axis that best explains the configuration:

    (t*, rho*) = argmin_{t, rho}  sum_i || refl(P[perm[i]]) - P[i] ||^2

The residual r_i = refl(P[perm[i]]) - P[i] is the displacement between a point and the
reflection of its partner. Its half-length |r_i| / 2 is the distance of point i from the
closest perfectly symmetric configuration (P + refl(P[perm])) / 2. We report
```

```
    a_i = |r_i| / (2 * IOD)          (per-point asymmetry, fraction of inter-ocular dist.)
    A   = mean_i a_i                 (global; over a chosen subset, e.g. the inner face)
    A_r = mean_{i in region r} a_i   (regional)

Because the axis is fitted, A is invariant to in-plane rotation, translation and scale
of the landmarks (verified in tests). This is the 2D analogue of the Procrustes
"object symmetry" decomposition of geometric morphometrics (Klingenberg et al. 2002),
restricted to a pure reflection.

Baseline measure (variant M0, re-implemented from nashokan/FacialAsymmetry)
---------------------------------------------------------------------------
Vertical midline x = c through the mean x of mesh vertices 33 and 263; every vertex left
of the line is reflected and matched to its nearest right-side vertex with a KD-tree;
score = mean matched distance / face width (|x_454 - x_234|).
"""
from __future__ import annotations

from dataclasses import dataclass

import numpy as np
from scipy.optimize import minimize
from scipy.spatial import cKDTree


# ---------------------------------------------------------------------------
# Reflection primitives
# ---------------------------------------------------------------------------
def reflect_points(P: np.ndarray, theta: float, rho: float) -> np.ndarray:
    """Reflect points P about the line n.x = rho with n = (cos theta, sin theta)."""
    n = np.array([np.cos(theta), np.sin(theta)])
    s = P @ n - rho
    return P - 2.0 * s[:, None] * n[None, :]


def reflect_vertical(P: np.ndarray, c: float) -> np.ndarray:
    """Reflect about the vertical line x = c, i.e. (x, y) -> (2c - x, y)."""
    Q = P.copy()
    Q[:, 0] = 2.0 * c - Q[:, 0]
    return Q


# ---------------------------------------------------------------------------
# Optimal symmetry axis
# ---------------------------------------------------------------------------
def _axis_init(P: np.ndarray, perm: np.ndarray) -> tuple[float, float]:
    """
    Closed-form initial guess. For each bilateral pair the vector between partners is
    (ideally) parallel to the axis normal, and their midpoint lies on the axis.
    """
    idx = np.where(perm != np.arange(len(perm)))[0]
    idx = idx[idx < perm[idx]]
    d = P[idx] - P[perm[idx]]
    # Orientation-free average of the pair directions (double-angle trick).
    ang = np.arctan2(d[:, 1], d[:, 0])
    w = np.linalg.norm(d, axis=1)
    theta = 0.5 * np.arctan2(np.sum(w * np.sin(2 * ang)), np.sum(w * np.cos(2 * ang)))
    n = np.array([np.cos(theta), np.sin(theta)])
    mids = 0.5 * (P[idx] + P[perm[idx]])
    rho = float(np.mean(mids @ n))
    return float(theta), rho


def fit_symmetry_axis(P: np.ndarray, perm: np.ndarray, weights: np.ndarray | None = None
                      ) -> tuple[float, float]:
    """Least-squares symmetry axis (theta, rho); see module docstring."""
    w = np.ones(len(P)) if weights is None else np.asarray(weights, float)
    Pp = P[perm]

    def cost(v):
        R = reflect_points(Pp, v[0], v[1]) - P
        return float(np.sum(w * np.sum(R * R, axis=1)))

    x0 = np.array(_axis_init(P, perm))
    res = minimize(cost, x0, method="Nelder-Mead",
                   options={"xatol": 1e-7, "fatol": 1e-9, "maxiter": 2000})
    return float(res.x[0]), float(res.x[1])


def axis_segment(theta: float, rho: float, P: np.ndarray) -> np.ndarray:
    """Two end points of the symmetry axis spanning the vertical extent of P (for drawing)."""
    n = np.array([np.cos(theta), np.sin(theta)])
    u = np.array([-n[1], n[0]])                    # direction along the axis
    c = rho * n                                    # closest point of the line to the origin
    t = (P - c) @ u
    return np.stack([c + u * (t.min() - 10), c + u * (t.max() + 10)])


# ---------------------------------------------------------------------------
# Proposed asymmetry measure
# ---------------------------------------------------------------------------
```

```
@dataclass
class AsymmetryResult:
    per_point: np.ndarray          # a_i, fraction of IOD, shape (n,)
    global_score: float            # mean a_i over `subset`
    region_scores: dict            # region name -> mean a_i
    theta: float                   # fitted axis
    rho: float
    iod: float                     # normaliser in pixels
    residuals: np.ndarray          # r_i in pixels, shape (n, 2)


def iod(P: np.ndarray, eye_pair: tuple[int, int]) -> float:
    """Inter-ocular distance between the two outer eye corners."""
    return float(np.linalg.norm(P[eye_pair[0]] - P[eye_pair[1]]))


def asymmetry(P: np.ndarray, perm: np.ndarray, regions: dict, subset: list[int],
              eye_pair: tuple[int, int], fit_on: list[int] | None = None) -> AsymmetryResult:
    """
    Compute per-point, global and regional asymmetry with a fitted symmetry axis.

    fit_on : indices used to fit the axis (default: `subset`). Using the inner face
             avoids letting the ill-defined jaw contour tilt the axis.
    """
    fit_on = subset if fit_on is None else fit_on
    w = np.zeros(len(P))
    w[fit_on] = 1.0
    # A pair only contributes if both partners are in the fitting set.
    w = w * w[perm]
    theta, rho = fit_symmetry_axis(P, perm, weights=w)
    R = reflect_points(P[perm], theta, rho) - P
    scale = iod(P, eye_pair)
    a = np.linalg.norm(R, axis=1) / (2.0 * scale)
    regs = {k: float(np.mean(a[v])) for k, v in regions.items()}
    return AsymmetryResult(a, float(np.mean(a[subset])), regs, theta, rho, scale, R)


def axis_tilt_deg(theta: float) -> float:
    """Angle between the fitted symmetry axis and the image vertical, in degrees."""
    # The normal of a vertical axis is horizontal (theta = 0 or pi).
    t = np.degrees(theta) % 180.0
    return float(t if t <= 90 else t - 180.0)


# ---------------------------------------------------------------------------------------
# Baseline measure (re-implementation of the public FacialAsymmetry method)
# ---------------------------------------------------------------------------------------
def baseline_kdtree_asymmetry(L: np.ndarray, mid_a: int = 33, mid_b: int = 263,
                              cheek_a: int = 234, cheek_b: int = 454,
                              margin: float = 1.5, perm: np.ndarray | None = None
                              ) -> dict:
    """
    Reflected nearest-neighbour matching on the dense mesh with a vertical midline.

    If `perm` is given, the KD-tree is replaced by the fixed mesh correspondence
    (used in the correspondence ablation); everything else is unchanged.
    Returns the global score (mean distance / face width) and the matched pairs.
    """
    c = 0.5 * (L[mid_a, 0] + L[mid_b, 0])
    width = abs(L[cheek_b, 0] - L[cheek_a, 0]) or 1.0
    left = np.where(L[:, 0] < c - margin)[0]
    right = np.where(L[:, 0] > c + margin)[0]
    if len(left) == 0 or len(right) == 0:
        return {"global_score": float("nan"), "pairs": np.zeros((0, 2), int),
                "distances": np.zeros(0), "midline_x": c, "face_width": width}
    refl_left = reflect_vertical(L[left], c)
    if perm is None:
        _, j = cKDTree(L[right]).query(refl_left)
        partners = right[j]
    else:
        partners = perm[left]
    d = np.linalg.norm(refl_left - L[partners], axis=1)
    return {"global_score": float(np.mean(d) / width),
            "pairs": np.stack([left, partners], 1), "distances": d,
            "midline_x": c, "face_width": width}


def mapping_cross_structure_rate(pairs: np.ndarray, perm: np.ndarray) -> float:
    """
    Fraction of KD-tree matches whose partner is NOT the anatomical mirror vertex.
    A diagnostic of how often nearest-neighbour matching pairs unrelated structures.
    """
    if len(pairs) == 0:
        return float("nan")
    return float(np.mean(perm[pairs[:, 0]] != pairs[:, 1]))


# ---------------------------------------------------------------------------------------
# Mirror-consistency (flip test)
# ---------------------------------------------------------------------------------------
def unflip_landmarks(Lf: np.ndarray, width: int, perm: np.ndarray) -> np.ndarray:
```

```
    """
    Map landmarks detected on a horizontally flipped image back to the original frame.

    MediaPipe returns continuous coordinates (x_norm * W, pixel centres at i + 0.5), so a
    point at x' in the flipped image is at W - x' in the original. Flipping also
    swaps anatomical sides, so the vertex detected as i in the flipped image is the
    original's vertex perm[i]; we therefore re-index with the mirror permutation.
    """
    U = Lf.copy()
    U[:, 0] = width - U[:, 0]
    out = np.empty_like(U)
    out[perm] = U
    return out


def mirror_inconsistency(L: np.ndarray, L_unflipped: np.ndarray, scale: float,
                         idx: list[int] | None = None) -> float:
    """Mean distance between the two detections, as a fraction of `scale` (e.g. IOD)."""
    idx = slice(None) if idx is None else idx
    return float(np.mean(np.linalg.norm(L[idx] - L_unflipped[idx], axis=1)) / scale)
```

src/fsm/detector.py

```
"""
detector.py -- thin wrapper around the MediaPipe Face Landmarker (Tasks API).

The model file models/face_landmarker.task is Google's published Face Landmarker
bundle (Apache-2.0). It is loaded once per process.
"""
from __future__ import annotations

import os
from dataclasses import dataclass
from functools import lru_cache

import cv2
import numpy as np

os.environ.setdefault("GLOG_minloglevel", "2")
import mediapipe as mp  # noqa: E402
from mediapipe.tasks import python as mp_python  # noqa: E402
from mediapipe.tasks.python import vision as mp_vision  # noqa: E402

from .mesh import ROOT  # noqa: E402

MODEL_PATH = os.path.join(ROOT, "models", "face_landmarker.task")
MODEL_URL = ("https://storage.googleapis.com/mediapipe-models/face_landmarker/"
             "face_landmarker/float16/1/face_landmarker.task")


@dataclass
class Detection:
    landmarks: np.ndarray      # (478, 2) pixel coordinates
    depth: np.ndarray          # (478,) relative depth (MediaPipe z, image-width units)
    transform: np.ndarray      # (4, 4) facial transformation matrix (canonical -> camera)
    n_faces: int               # number of faces found (we analyse the first)


@lru_cache(maxsize=1)
def _landmarker():
    if not os.path.exists(MODEL_PATH):
        raise FileNotFoundError(
            f"Model not found at {MODEL_PATH}. Download it from {MODEL_URL}")
    opts = mp_vision.FaceLandmarkerOptions(
        base_options=mp_python.BaseOptions(model_asset_path=MODEL_PATH),
        running_mode=mp_vision.RunningMode.IMAGE,
        num_faces=2,                                # 2 so that "multiple faces" can be reported
        min_face_detection_confidence=0.5,
        min_face_presence_confidence=0.5,
        output_face_blendshapes=False,
        output_facial_transformation_matrixes=True,   # used for yaw / pitch / roll
    )
    return mp_vision.FaceLandmarker.create_from_options(opts)


def detect(img_bgr: np.ndarray) -> Detection | None:
    """Run the landmarker on a BGR image. Returns None if no face is found."""
    if img_bgr is None or img_bgr.size == 0:
        return None
    if img_bgr.ndim == 2:
        img_bgr = cv2.cvtColor(img_bgr, cv2.COLOR_GRAY2BGR)
    rgb = np.ascontiguousarray(cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB))
    res = _landmarker().detect(mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb))
    if not res.face_landmarks:
        return None
    h, w = img_bgr.shape[:2]
    face = res.face_landmarks[0]
    xy = np.array([[p.x * w, p.y * h] for p in face], dtype=np.float64)
    z = np.array([p.z for p in face], dtype=np.float64)
    T = (np.asarray(res.facial_transformation_matrixes[0], dtype=np.float64)
```

```
        if res.facial_transformation_matrixes else np.eye(4))
    return Detection(xy, z, T, len(res.face_landmarks))


def pose_from_transform(T: np.ndarray) -> dict:
    """
    Yaw / pitch / roll in degrees from the rotation part of the facial transformation
    matrix (rotation of the canonical face model into camera space, ZYX convention).
    Signs depend on the camera convention; the gate only uses magnitudes for yaw and
    pitch. Validated in tests by the flip test (yaw and roll change sign under flip).
    """
    R = T[:3, :3]
    sy = np.hypot(R[0, 0], R[1, 0])
    pitch = np.degrees(np.arctan2(R[2, 1], R[2, 2]))
    yaw = np.degrees(np.arctan2(-R[2, 0], sy))
    roll = np.degrees(np.arctan2(R[1, 0], R[0, 0]))
    return {"yaw": float(yaw), "pitch": float(pitch), "roll": float(roll)}
```

src/fsm/quality.py

```
"""
quality.py -- observable image-quality signals and the accept / caution / reject gate.

Every signal is something the system can actually measure on the input; we do not
invent a per-landmark confidence (MediaPipe's Face Landmarker returns none: its
`visibility` and `presence` fields are None).

Signals
-------
iod_px          inter-ocular distance in pixels (face size)
sharpness       log10 variance of the Laplacian of the face crop resized to 256 px width
brightness      mean luma (0-255) of the face crop
clipped         fraction of face-crop pixels that are near-black (<16) or near-white (>239)
yaw, pitch      head rotation magnitudes (degrees) from the facial transformation matrix
mirror_incons   mean |L - unflip(detect(flip(I)))| / IOD on the inner face (proposed)
n_faces         number of faces found

Thresholds are stored in configs/gate_thresholds.json. Pose and size limits are set a
priori from the measurement's assumptions; image-statistic limits are calibrated on the
LFPW *training* split by experiments/calibrate_gate.py (never on the test split).
"""
from __future__ import annotations

import json
import os

import cv2
import numpy as np

from .mesh import CONFIG_DIR

GATE_PATH = os.path.join(CONFIG_DIR, "gate_thresholds.json")

# Defaults (overwritten by the calibrated file when present).
# direction "max": larger is worse;  "min": smaller is worse.
DEFAULT_GATE = {
    "iod_px":        {"direction": "min", "caution": 60.0, "reject": 30.0},
    "sharpness":     {"direction": "min", "caution": 1.6, "reject": 1.0},
    "brightness_lo": {"direction": "min", "caution": 60.0, "reject": 35.0},
    "brightness_hi": {"direction": "max", "caution": 200.0, "reject": 225.0},
    "clipped":       {"direction": "max", "caution": 0.10, "reject": 0.25},
    "abs_yaw":       {"direction": "max", "caution": 10.0, "reject": 20.0},
    "abs_pitch":     {"direction": "max", "caution": 15.0, "reject": 25.0},
    "mirror_incons": {"direction": "max", "caution": 0.035, "reject": 0.06},
}

# Plain-language explanations shown in the app.
EXPLAIN = {
    "iod_px": "face is small in the image (few pixels between the eyes)",
    "sharpness": "face region looks blurred / out of focus",
    "brightness_lo": "face region is under-exposed (too dark)",
    "brightness_hi": "face region is over-exposed (too bright)",
    "clipped": "many face pixels are clipped to black or white",
    "abs_yaw": "head is turned left/right; 2D left-right comparison is distorted",
    "abs_pitch": "head is tilted up/down; vertical proportions are distorted",
    "mirror_incons": "landmarks are not reproducible on the mirrored image "
                     "(detector is unsure of the geometry)",
    "n_faces": "more than one face found; the first face was analysed",
}


def load_gate() -> dict:
    if os.path.exists(GATE_PATH):
        with open(GATE_PATH) as f:
            return json.load(f)["thresholds"]
    return DEFAULT_GATE


def face_crop(img: np.ndarray, pts: np.ndarray, margin: float = 0.15) -> np.ndarray:
    """Axis-aligned crop around the landmarks with a relative margin, clipped to the image."""
```

```
    h, w = img.shape[:2]
    x0, y0 = pts.min(0)
    x1, y1 = pts.max(0)
    mx, my = margin * (x1 - x0), margin * (y1 - y0)
    x0, y0 = int(max(0, x0 - mx)), int(max(0, y0 - my))
    x1, y1 = int(min(w, x1 + mx)), int(min(h, y1 + my))
    if x1 <= x0 + 2 or y1 <= y0 + 2:
        return img
    return img[y0:y1, x0:x1]


def image_quality(img: np.ndarray, pts: np.ndarray) -> dict:
    """Sharpness / exposure statistics of the face region (independent of the detector)."""
    crop = face_crop(img, pts)
    gray = cv2.cvtColor(crop, cv2.COLOR_BGR2GRAY) if crop.ndim == 3 else crop
    # Resize to a fixed width so that the focus measure does not depend on face size.
    scale = 256.0 / max(1, gray.shape[1])
    g = cv2.resize(gray, None, fx=scale, fy=scale, interpolation=cv2.INTER_AREA)
    lap_var = float(cv2.Laplacian(g, cv2.CV_64F).var())
    return {
        "sharpness": float(np.log10(lap_var + 1e-6)),
        "brightness": float(g.mean()),
        "clipped": float(np.mean((g < 16) | (g > 239))),
    }


def gate_decision(signals: dict, gate: dict | None = None, use_mirror: bool = True
                  ) -> tuple[str, list[dict]]:
    """
    Combine signals into accept / caution / reject. The worst signal decides.
    Returns (status, reasons) where each reason names the signal, value and level.
    """
    gate = load_gate() if gate is None else gate
    vals = dict(signals)
    vals["brightness_lo"] = vals.get("brightness")
    vals["brightness_hi"] = vals.get("brightness")
    order = {"accept": 0, "caution": 1, "reject": 2}
    status, reasons = "accept", []
    for name, rule in gate.items():
        if name == "mirror_incons" and not use_mirror:
            continue
        v = vals.get(name)
        if v is None or not np.isfinite(v):
            continue
        bad = (lambda t: v > t) if rule["direction"] == "max" else (lambda t: v < t)
        level = "reject" if bad(rule["reject"]) else "caution" if bad(rule["caution"]) else None
        if level:
            reasons.append({"signal": name, "value": float(v), "level": level,
                            "why": EXPLAIN.get(name, name)})
            if order[level] > order[status]:
                status = level
    if signals.get("n_faces", 1) > 1:
        reasons.append({"signal": "n_faces", "value": signals["n_faces"], "level": "caution",
                        "why": EXPLAIN["n_faces"]})
        status = max(status, "caution", key=order.get)
    return status, reasons


def risk_score(signals: dict, gate: dict | None = None, use_mirror: bool = True) -> float:
    """
    Continuous risk used for risk-coverage curves: for each signal, how far it is past its
    caution threshold in units of the (caution -> reject) interval; the maximum is taken.
    0 = at the caution threshold, 1 = at the reject threshold, negative = comfortably fine.
    """
    gate = load_gate() if gate is None else gate
    vals = dict(signals)
    vals["brightness_lo"] = vals.get("brightness")
    vals["brightness_hi"] = vals.get("brightness")
    out = []
    for name, rule in gate.items():
        if name == "mirror_incons" and not use_mirror:
            continue
        v = vals.get(name)
        if v is None or not np.isfinite(v):
            continue
        span = (rule["reject"] - rule["caution"]) or 1e-9
        out.append((v - rule["caution"]) / span)   # sign handles both directions
    return float(max(out)) if out else 0.0
```

src/fsm/pipeline.py

```
"""
pipeline.py -- end-to-end measurement. Callable from experiments and from the app.

`measure(img)` runs the detector at most three times (original, mirrored, and the
baseline's aligned re-detection) and returns every quantity needed by the four
compared variants, so experiments evaluate all variants on exactly the same detections:

  M0  Baseline     re-implementation of nashokan/FacialAsymmetry: eye-corner affine
                   alignment + re-detection, vertical midline, KD-tree reflected
```

```
                  nearest-neighbour matching on the 478-point mesh, / face width.
  M1  Fixed-pair  fixed anatomical correspondence on the 68-point layout, fitted
                  symmetry axis (rotation invariant, no re-detection), / IOD.
  M2  +Gate       M1 + accept/caution/reject gate on size, blur, exposure, pose.
  M3  +Mirror     M2 with mirror-consistent landmarks: the landmark set is the average
                  of the detection on I and the un-flipped detection on flip(I); the
                  mirror inconsistency is added to the gate and used for per-region
                  validity.  (Proposed full system.)
"""
from __future__ import annotations

import time

import cv2
import numpy as np

from . import geometry as G
from . import mesh as MS
from .detector import detect, pose_from_transform
from .quality import gate_decision, image_quality, load_gate, risk_score

PERM68 = MS.ibug68_permutation()
PERM478 = MS.mp_permutation()

# Per-region mirror-inconsistency limit (fraction of IOD) above which a region is marked
# "unreliable" in M3. Calibrated on the training split (calibrate_gate.py writes it into
# gate_thresholds.json under "region_mirror_limit").
DEFAULT_REGION_LIMIT = 0.05


def align_like_baseline(img: np.ndarray, L: np.ndarray, size=(600, 600)) -> np.ndarray:
    """Baseline alignment: similarity transform taking vertices 33, 263 to fixed positions."""
    w, h = size
    src = np.float32([L[33], L[263]])
    dst = np.float32([[w * 0.33, h * 0.33], [w * 0.67, h * 0.33]])
    M, _ = cv2.estimateAffinePartial2D(src, dst)
    return img if M is None else cv2.warpAffine(img, M, size)


def asym68(P68: np.ndarray) -> G.AsymmetryResult:
    """Proposed measure on a 68-point configuration (axis fitted on the inner face)."""
    return G.asymmetry(P68, PERM68, MS.IBUG68_REGIONS, subset=MS.IBUG68_INNER,
                       eye_pair=MS.IBUG68_OUTER_EYE, fit_on=MS.IBUG68_INNER)


def raw_detect(img: np.ndarray, run_baseline: bool = True, run_mirror: bool = True) -> dict:
    """
    All detector calls for one image (original, mirrored, baseline re-detection).
    Kept separate from the geometry so that experiments can cache detections.
    """
    t0 = time.perf_counter()
    raw: dict = {"ok": False, "failure": None}
    if img is None or img.size == 0:
        raw["failure"] = "unreadable image"
        return raw
    det = detect(img)
    if det is None:
        raw["failure"] = "no face detected"
        return raw
    raw.update(ok=True, width=img.shape[1], height=img.shape[0], L=det.landmarks,
               T=det.transform, n_faces=det.n_faces,
               quality=image_quality(img, det.landmarks[:468]))
    if run_mirror:
        detf = detect(cv2.flip(img, 1))
        raw["Lf"] = None if detf is None else detf.landmarks       # in flipped-image frame
    if run_baseline:
        det2 = detect(align_like_baseline(img, det.landmarks))
        raw["L_aligned"] = None if det2 is None else det2.landmarks
    raw["detect_s"] = time.perf_counter() - t0
    return raw


def measure_from_raw(raw: dict) -> dict:
    """Geometry for every variant from cached detections (no detector calls)."""
    out: dict = {"ok": raw.get("ok", False), "failure": raw.get("failure")}
    if not out["ok"]:
        return out
    L = raw["L"]
    P = MS.mp_to_68(L)
    out.update(n_faces=raw["n_faces"], lm478=L, lm68=P, transform=raw["T"],
               pose=pose_from_transform(raw["T"]), quality=raw["quality"],
               iod_px=G.iod(P, MS.IBUG68_OUTER_EYE), single=asym68(P))

    # ---- mirror-consistency (M3) ----------------------------------------------------
    if "Lf" in raw:
        if raw["Lf"] is None:
            out["mirror"] = None
        else:
            U = G.unflip_landmarks(raw["Lf"], raw["width"], PERM478)
            Pu = MS.mp_to_68(U)
            Pm = 0.5 * (P + Pu)
```

```
            inner = MS.IBUG68_INNER
            out["mirror"] = {
                "lm478_unflipped": U, "lm68_unflipped": Pu, "lm68_avg": Pm,
                "lm478_avg": 0.5 * (L + U),
                "incons": G.mirror_inconsistency(P, Pu, out["iod_px"], inner),
                "region_incons": {k: G.mirror_inconsistency(P, Pu, out["iod_px"], v)
                                  for k, v in MS.IBUG68_REGIONS.items()},
                "score_flip": asym68(Pu).global_score,
                "avg": asym68(Pm),
            }

    # ---- baseline (M0) ------------------------------------------------------------
    if "L_aligned" in raw:
        if raw["L_aligned"] is None:
            out["baseline"] = None
        else:
            La = raw["L_aligned"]
            b = G.baseline_kdtree_asymmetry(La)
            b["cross_structure_rate"] = G.mapping_cross_structure_rate(b["pairs"], PERM478)
            b["fixed_pair_score"] = G.baseline_kdtree_asymmetry(La, perm=PERM478)["global_score"]
            # Same KD-tree measure on the *un-aligned* detection (rotation study)
            b["noalign_score"] = G.baseline_kdtree_asymmetry(L)["global_score"]
            b["lm478_aligned"] = La
            out["baseline"] = b
    out["detect_s"] = raw.get("detect_s")
    return out


def measure(img: np.ndarray, run_baseline: bool = True, run_mirror: bool = True) -> dict:
    """Measure everything for one BGR image. Never raises on a bad image: reports failure."""
    return measure_from_raw(raw_detect(img, run_baseline, run_mirror))


def signals_of(m: dict, use_mirror: bool) -> dict:
    """Collect the gate signals of a measurement."""
    s = {"iod_px": m["iod_px"], **m["quality"],
         "abs_yaw": abs(m["pose"]["yaw"]), "abs_pitch": abs(m["pose"]["pitch"]),
         "n_faces": m["n_faces"]}
    if use_mirror:
        s["mirror_incons"] = (m["mirror"]["incons"] if m.get("mirror") else np.inf)
    return s


def variant_outputs(m: dict, gate: dict | None = None) -> dict:
    """
    Turn one measurement into the four variants' user-facing outputs:
    score (or NaN if not produced), status, risk and region scores.
    """
    gate_cfg = load_gate() if gate is None else gate
    region_limit = DEFAULT_REGION_LIMIT
    try:
        import json, os
        with open(os.path.join(MS.CONFIG_DIR, "gate_thresholds.json")) as f:
            region_limit = json.load(f).get("region_mirror_limit", DEFAULT_REGION_LIMIT)
    except Exception:
        pass
    v = {}
    if not m.get("ok"):
        for k in ("M0", "M1", "M2", "M3"):
            v[k] = {"score": np.nan, "status": "reject", "risk": np.inf, "regions": {}}
        return v
    b = m.get("baseline")
    v["M0"] = {"score": b["global_score"] if b else np.nan, "status": "accept",
               "risk": 0.0, "regions": {}}
    s1 = m["single"]
    v["M1"] = {"score": s1.global_score, "status": "accept", "risk": 0.0,
               "regions": dict(s1.region_scores)}
    sig2 = signals_of(m, use_mirror=False)
    st2, why2 = gate_decision(sig2, gate_cfg, use_mirror=False)
    v["M2"] = {"score": s1.global_score, "status": st2, "reasons": why2,
               "risk": risk_score(sig2, gate_cfg, use_mirror=False),
               "regions": dict(s1.region_scores)}
    mir = m.get("mirror")
    sig3 = signals_of(m, use_mirror=True)
    st3, why3 = gate_decision(sig3, gate_cfg, use_mirror=True)
    if mir is None:
        st3 = "reject"
        why3 = why3 + [{"signal": "mirror", "value": np.nan, "level": "reject",
                        "why": "no face found in the mirrored image"}]
        v["M3"] = {"score": np.nan, "status": st3, "reasons": why3, "risk": np.inf,
                   "regions": {}, "region_valid": {}}
    else:
        a = mir["avg"]
        valid = {k: bool(mir["region_incons"][k] <= region_limit) for k in a.region_scores}
        v["M3"] = {"score": a.global_score, "status": st3, "reasons": why3,
                   "risk": risk_score(sig3, gate_cfg, use_mirror=True),
                   "regions": dict(a.region_scores), "region_valid": valid,
                   "stability_band": abs(m["single"].global_score - mir["score_flip"])}
    return v
```

```
def symmetry_index(A: float, alpha: float = 10.0) -> float:
    """
    Optional readable 0-100 index, S = 100 / (1 + alpha * 100 * A), A as fraction of IOD.
    alpha is chosen so that the LFPW-train median maps near 50 (see calibrate_gate.py).
    This is a monotone re-expression of A, NOT a validated clinical or aesthetic score.
    """
    return float(100.0 / (1.0 + alpha * 100.0 * A)) if np.isfinite(A) else float("nan")
```

src/fsm/data.py

```
"""
data.py -- reading the LFPW (300W subset) images and 68-point annotations.

Expected layout (see datasets/README.md for how to obtain the data):
    datasets/lfpw/trainset/image_XXXX.png + image_XXXX.pts   (811 images)
    datasets/lfpw/testset/image_XXXX.png  + image_XXXX.pts   (224 images)
The .pts files follow the iBUG format: a header, then 68 "x y" lines inside braces.
iBUG coordinates are 1-based (Matlab convention); we subtract 1 to obtain 0-based pixels.
"""
from __future__ import annotations

import glob
import os

import numpy as np

from .mesh import ROOT

DATA_ROOT = os.environ.get("FSM_DATA", os.path.join(ROOT, "datasets", "lfpw"))
SPLITS = {"train": "trainset", "test": "testset"}


def read_pts(path: str) -> np.ndarray:
    """Parse an iBUG .pts file into a (68, 2) float array of 0-based pixel coordinates."""
    with open(path) as f:
        lines = [ln.strip() for ln in f]
    start = lines.index("{") + 1
    end = lines.index("}")
    pts = np.array([list(map(float, ln.split())) for ln in lines[start:end]])
    assert pts.shape == (68, 2), f"{path}: expected 68 points, got {pts.shape}"
    return pts - 1.0


def list_split(split: str) -> list[tuple[str, str]]:
    """Sorted list of (image_path, pts_path) for a split ('train' or 'test')."""
    d = os.path.join(DATA_ROOT, SPLITS[split])
    items = []
    for img in sorted(glob.glob(os.path.join(d, "*.png"))):
        pts = os.path.splitext(img)[0] + ".pts"
        if os.path.exists(pts):
            items.append((img, pts))
    return items


def crop_to_annotation(img, gt68, expand: float = 0.6):
    """
    Crop the image around the annotated face (benchmark protocol: 300W provides a face
    box per image). Many LFPW images contain several people; without this crop the
    detector may analyse a different person than the one annotated.
    The box is the landmark extent enlarged by `expand` on every side (relative to its
    size), clipped to the image. Returns (crop, gt68 shifted into crop coordinates).
    """
    h, w = img.shape[:2]
    x0, y0 = gt68.min(0)
    x1, y1 = gt68.max(0)
    bw, bh = x1 - x0, y1 - y0
    X0, Y0 = int(max(0, x0 - expand * bw)), int(max(0, y0 - expand * bh))
    X1, Y1 = int(min(w, x1 + expand * bw)), int(min(h, y1 + expand * bh))
    return img[Y0:Y1, X0:X1].copy(), gt68 - np.array([X0, Y0], dtype=float)
```

src/fsm/viz.py

```
"""
viz.py -- drawing helpers shared by the Streamlit app and the paper figures.
All functions take a BGR image and return a new BGR image (input is never modified).
"""
from __future__ import annotations

import cv2
import numpy as np

from . import geometry as G
from . import mesh as MS

REGION_COLORS = {          # BGR, colour-blind-friendly (Okabe-Ito)
    "Brows": (0, 159, 230),
    "Eyes": (233, 180, 86),
    "Nose": (115, 158, 0),
```

```
    "Mouth": (0, 94, 213),
    "Jaw": (167, 121, 204),
}
STATUS_COLORS = {"accept": (80, 160, 40), "caution": (0, 170, 240), "reject": (40, 40, 220)}


def _thick(img):
    return max(1, int(round(img.shape[1] / 400)))


def draw_mesh(img, L, color=(255, 255, 255), radius=None):
    out = img.copy()
    r = radius or max(1, _thick(img) - 1)
    for x, y in L:
        cv2.circle(out, (int(x), int(y)), r, color, -1, cv2.LINE_AA)
    return out


def draw_axis(img, res: G.AsymmetryResult, P, color=(255, 0, 255)):
    out = img.copy()
    a, b = G.axis_segment(res.theta, res.rho, P)
    cv2.line(out, tuple(int(v) for v in a), tuple(int(v) for v in b), color,
             _thick(img) + 1, cv2.LINE_AA)
    return out


def draw_pairs(img, P, perm=None, color=(255, 255, 0)):
    """Connect each bilateral pair of the 68-point layout."""
    out = img.copy()
    perm = MS.ibug68_permutation() if perm is None else perm
    for i, j in enumerate(perm):
        if i < j and i in MS.IBUG68_INNER:
            cv2.line(out, tuple(P[i].astype(int)), tuple(P[j].astype(int)), color, 1, cv2.LINE_AA)
    return out


def draw_mirrored(img, res: G.AsymmetryResult, P):
    """Original points (green) and reflections of their partners (red), joined by residuals."""
    out = img.copy()
    perm = MS.ibug68_permutation()
    Q = G.reflect_points(P[perm], res.theta, res.rho)
    t = _thick(img)
    for i in MS.IBUG68_INNER:
        cv2.line(out, tuple(P[i].astype(int)), tuple(Q[i].astype(int)), (128, 128, 128), 1,
                 cv2.LINE_AA)
        cv2.circle(out, tuple(P[i].astype(int)), t + 1, (60, 200, 60), -1, cv2.LINE_AA)
        cv2.circle(out, tuple(Q[i].astype(int)), t + 1, (60, 60, 230), -1, cv2.LINE_AA)
    return out


def draw_heatmap(img, res: G.AsymmetryResult, P, vmax=0.06, valid: dict | None = None):
    """Colour each 68-point by its per-point asymmetry (fraction of IOD); greys out invalid regions."""
    out = (0.55 * img).astype(np.uint8) if img.mean() < 200 else img.copy()
    cmap = cv2.applyColorMap(np.uint8(np.clip(np.linspace(0, 1, 256) * 255, 0, 255))[:, None],
                             cv2.COLORMAP_INFERNO)[:, 0, :]
    t = _thick(img)
    region_of = {i: k for k, v in MS.IBUG68_REGIONS.items() for i in v}
    for i, (x, y) in enumerate(P):
        reg = region_of[i]
        if valid is not None and not valid.get(reg, True):
            col = (140, 140, 140)
        else:
            col = tuple(int(c) for c in cmap[int(np.clip(res.per_point[i] / vmax, 0, 1) * 255)])
        cv2.circle(out, (int(x), int(y)), 2 * t + 2, col, -1, cv2.LINE_AA)
    return out


def draw_regions(img, P):
    out = img.copy()
    t = _thick(img)
    for k, idx in MS.IBUG68_REGIONS.items():
        for i in idx:
            cv2.circle(out, tuple(P[i].astype(int)), 2 * t + 1, REGION_COLORS[k], -1, cv2.LINE_AA)
    return out


def draw_kdtree_matches(img, L, pairs, perm, max_lines=400):
    """Baseline KD-tree matches: correct anatomical partner (green) vs wrong partner (red)."""
    out = img.copy()
    for li, ri in pairs[:max_lines]:
        col = (60, 200, 60) if perm[li] == ri else (60, 60, 230)
        cv2.line(out, tuple(L[li].astype(int)), tuple(L[ri].astype(int)), col, 1, cv2.LINE_AA)
    return out


def status_banner(img, status, text=None):
    out = img.copy()
    h = max(24, img.shape[0] // 14)
    cv2.rectangle(out, (0, 0), (img.shape[1], h), STATUS_COLORS[status], -1)
    cv2.putText(out, (text or status.upper()), (8, int(h * 0.72)), cv2.FONT_HERSHEY_SIMPLEX,
                h / 40, (255, 255, 255), max(1, h // 18), cv2.LINE_AA)
```

```
    return out
```

app.py

```
"""
app.py -- Streamlit demo of the quality-aware facial symmetry measurement system.

    streamlit run app.py

The UI only *calls* the tested engine in src/fsm; no geometry is implemented here.
"""
import io
import json
import os
import sys

import cv2
import numpy as np
import pandas as pd
import streamlit as st

sys.path.insert(0, os.path.join(os.path.dirname(__file__), "src"))
from fsm import viz  # noqa: E402
from fsm.mesh import CONFIG_DIR, IBUG68_REGIONS  # noqa: E402
from fsm.pipeline import measure, signals_of, variant_outputs  # noqa: E402
from fsm.quality import EXPLAIN, load_gate  # noqa: E402

st.set_page_config(page_title="Face Symmetry Measurement", layout="wide")

with open(os.path.join(CONFIG_DIR, "gate_thresholds.json")) as f:
    CFG = json.load(f)
ALPHA = CFG.get("index_alpha_per_pct", 0.5)
STATUS_TEXT = {
    "accept": ("[OK] ACCEPT", "Image meets the measurement conditions. The score below is reported."),
    "caution": ("[!] CAUTION", "The score is shown, but at least one condition is outside the "
                "calibrated range. Interpret with care."),
    "reject": ("[X] REJECT", "The image does not meet the measurement conditions, so no score "
               "is reported. Fix the issues listed and try again."),
}


# ---------------------------------------------------------------- helpers
def to_bgr(file_bytes):
    arr = np.frombuffer(file_bytes, np.uint8)
    return cv2.imdecode(arr, cv2.IMREAD_COLOR)


def limit_size(img, max_side=1280):
    """Large phone photos are downscaled for speed (does not change the normalised score)."""
    s = max_side / max(img.shape[:2])
    return cv2.resize(img, None, fx=s, fy=s, interpolation=cv2.INTER_AREA) if s < 1 else img


def perturb(img, brightness, blur, angle, occlude):
    """Optional controlled perturbations so the demo can show the gate reacting live."""
    out = np.clip(img.astype(np.float32) * brightness, 0, 255).astype(np.uint8)
    if blur > 0:
        out = cv2.GaussianBlur(out, (0, 0), blur)
    if angle != 0:
        h, w = out.shape[:2]
        M = cv2.getRotationMatrix2D((w / 2, h / 2), angle, 1.0)
        out = cv2.warpAffine(out, M, (w, h), borderMode=cv2.BORDER_REPLICATE)
    if occlude != "none":
        h, w = out.shape[:2]
        box = {"left half of image": (0, 0, w // 2, h),
               "upper-left quarter": (0, 0, w // 2, h // 2),
               "lower third": (0, 2 * h // 3, w, h)}[occlude]
        cv2.rectangle(out, box[:2], box[2:], (128, 128, 128), -1)
    return out


def rgb(img):
    return cv2.cvtColor(img, cv2.COLOR_BGR2RGB)


def index_of(A_pct):
    return 100.0 / (1.0 + ALPHA * A_pct)


# ---------------------------------------------------------------- sidebar
st.sidebar.header("Input")
source = st.sidebar.radio("Image source", ["Upload a photo", "Webcam snapshot"])
st.sidebar.header("Stress-test the input (demo)")
st.sidebar.caption("Apply a controlled change and watch the quality gate respond.")
b = st.sidebar.slider("Brightness factor", 0.2, 2.5, 1.0, 0.05)
bl = st.sidebar.slider("Gaussian blur sigma (px)", 0.0, 15.0, 0.0, 0.5)
ang = st.sidebar.slider("In-plane rotation (deg)", -40, 40, 0, 1)
occ = st.sidebar.selectbox("Occlusion", ["none", "upper-left quarter", "left half of image",
                                         "lower third"])
show_base = st.sidebar.checkbox("Compare with baseline (M0)", value=True)
```

```
# ------------------------------------------------------------------ header
st.title("Face Symmetry Measurement")
st.markdown(
    "Quality-aware, region-based **2D geometric** facial asymmetry. The system checks whether "
    "the photo is suitable *before* reporting a number, and verifies its own landmarks with a "
    "mirror test.  \n"
    "<small>This is an image-based geometric indicator only: not a medical assessment and not "
    "a measure of attractiveness. Photos are processed in memory and not stored.</small>",
    unsafe_allow_html=True)

file_bytes = None
if source == "Upload a photo":
    up = st.file_uploader("Upload one clear, frontal face photo (JPG/PNG)",
                          type=["jpg", "jpeg", "png", "bmp", "webp"])
    file_bytes = up.getvalue() if up else None
else:
    cam = st.camera_input("Look straight at the camera, neutral expression")
    file_bytes = cam.getvalue() if cam else None

if file_bytes is None:
    st.info("Tips for a valid measurement: face the camera, keep the head level, even "
            "lighting, neutral expression, nothing covering the face.")
    st.stop()

img0 = to_bgr(file_bytes)
if img0 is None:
    st.error("Could not read this file as an image.")
    st.stop()
img = perturb(limit_size(img0), b, bl, ang, occ)

with st.spinner("Detecting landmarks (original + mirrored image)..."):
    m = measure(img, run_baseline=show_base, run_mirror=True)

if not m["ok"]:
    st.error(f"[X] REJECT: {m['failure']}. Please upload a photo with one clearly visible face.")
    st.image(rgb(img), caption="input", width=360)
    st.stop()

v = variant_outputs(m)
out = v["M3"]
status = out["status"]
label, expl = STATUS_TEXT[status]

# ------------------------------------------------------------------ verdict + numbers
c1, c2 = st.columns([1.1, 1])
with c1:
    {"accept": st.success, "caution": st.warning, "reject": st.error}[status](f"**{label}**: {expl}")
    if out.get("reasons"):
        for r in out["reasons"]:
            st.markdown(f"- **{r['level']}**: {r['why']} "
                        f"(`{r['signal']}` = {r['value']:.3g})")
with c2:
    A = 100 * out["score"] if np.isfinite(out["score"]) else np.nan
    band = 100 * out.get("stability_band", np.nan)
    show_score = status != "reject" or st.checkbox("Show the (unreliable) score anyway")
    k1, k2, k3 = st.columns(3)
    if show_score and np.isfinite(A):
        k1.metric("Asymmetry A (% IOD)", f"{A:.2f}", help="Mean distance of the inner-face "
                  "landmarks from the closest perfectly mirror-symmetric shape, as a percentage "
                  "of the distance between the outer eye corners. Lower = more symmetric.")
        k2.metric("Mirror stability +/- (% IOD)", f"{band:.2f}", help="|A(original) - A(mirrored "
                  "image)|. A perfectly consistent detector gives 0.")
        k3.metric("Symmetry index (0-100)", f"{index_of(A):.0f}", help="S = 100/(1+alpha*A); alpha set so "
                  "the median LFPW-train face scores 50. A readable re-expression of A, not a "
                  "validated clinical or aesthetic score.")
    else:
        k1.metric("Asymmetry A", "withheld")
    if show_base and v["M0"]["score"] == v["M0"]["score"]:
        st.caption(f"Baseline M0 (KD-tree, vertical midline): {100 * v['M0']['score']:.2f} % of "
                   "face width, reported without any quality check.")

# ------------------------------------------------------------------ visualisations
mir = m["mirror"]
res = mir["avg"] if mir else m["single"]
P = mir["lm68_avg"] if mir else m["lm68"]
valid = out.get("region_valid", {})
tabs = st.tabs(["Overlays", "Regions", "Quality signals", "Export"])


def zoom(im, pts=m["lm478"][:468], margin=0.25):
    """Crop a drawn overlay to the face so the landmarks are visible."""
    x0, y0 = pts.min(0)
    x1, y1 = pts.max(0)
    mx, my = margin * (x1 - x0), margin * (y1 - y0)
    h, w = im.shape[:2]
    return im[int(max(0, y0 - my)):int(min(h, y1 + my)), int(max(0, x0 - mx)):int(min(w, x1 + mx))]


with tabs[0]:
    a, b2, c, d = st.columns(4)
```

```
    a.image(rgb(zoom(viz.draw_mesh(img, m["lm478"][:468]))), caption="478-point mesh",
            use_container_width=True)
    b2.image(rgb(zoom(viz.draw_axis(viz.draw_pairs(img, P), res, P))),
             caption="fitted symmetry axis + bilateral pairs", use_container_width=True)
    c.image(rgb(zoom(viz.draw_mirrored(img, res, P))),
            caption="green: landmark, red: reflection of its partner", use_container_width=True)
    d.image(rgb(zoom(viz.draw_heatmap(img, res, P, valid=valid))),
            caption="per-point asymmetry (bright = larger; grey = unreliable region)",
            use_container_width=True)

with tabs[1]:
    rows = []
    for k in IBUG68_REGIONS:
        ok_r = valid.get(k, True)
        rows.append({"region": k, "asymmetry (% IOD)": round(100 * res.region_scores[k], 2),
                     "mirror inconsistency (% IOD)":
                         round(100 * mir["region_incons"][k], 2) if mir else None,
                     "status": "reliable" if ok_r else "unreliable (detector inconsistent)"})
    rdf = pd.DataFrame(rows)
    st.dataframe(rdf, hide_index=True, use_container_width=True)
    inner = rdf[(rdf.region != "Jaw") & (rdf.status == "reliable")]
    if len(inner):
        top = inner.sort_values("asymmetry (% IOD)", ascending=False).iloc[0]
        st.markdown(f"Largest measured inner-face asymmetry: **{top['region']}** "
                    f"({top['asymmetry (% IOD)']:.2f} % IOD).")
    st.bar_chart(rdf.set_index("region")["asymmetry (% IOD)"], horizontal=True)
    st.caption("The jaw contour is a silhouette whose landmark positions shift with head pose; "
               "its values are larger and less reliable, so it is excluded from the global score.")

with tabs[2]:
    gate = load_gate()
    sig = signals_of(m, use_mirror=True)
    sig["brightness_lo"] = sig["brightness_hi"] = sig["brightness"]
    rows = []
    for name, rule in gate.items():
        rows.append({"signal": name, "value": round(float(sig[name]), 4),
                     "caution if": f"{'<' if rule['direction'] == 'min' else '>'} {rule['caution']:.3g}",
                     "reject if": f"{'<' if rule['direction'] == 'min' else '>'} {rule['reject']:.3g}",
                     "meaning": EXPLAIN.get(name, "")})
    st.dataframe(pd.DataFrame(rows), hide_index=True, use_container_width=True)
    p = m["pose"]
    st.caption(f"Pose estimate: yaw {p['yaw']:.1f} deg, pitch {p['pitch']:.1f} deg, roll {p['roll']:.1f} deg "
               "(roll is handled by the fitted symmetry axis and is not gated).")

with tabs[3]:
    record = {
        "status": status, "reasons": out.get("reasons", []),
        "asymmetry_pct_iod": None if status == "reject" else A,
        "mirror_stability_pct_iod": band, "symmetry_index": None if status == "reject" else index_of(A),
        "regions_pct_iod": {k: 100 * x for k, x in res.region_scores.items()},
        "region_valid": valid, "pose_deg": m["pose"], "quality": m["quality"],
        "iod_px": m["iod_px"], "baseline_M0_pct_face_width":
            100 * v["M0"]["score"] if show_base else None,
        "landmarks68_xy": P.round(2).tolist(),
    }
    st.download_button("Download JSON", json.dumps(record, indent=1, default=float),
                       "symmetry_result.json", "application/json")
    flat = {k: val for k, val in record.items() if not isinstance(val, (dict, list))}
    flat.update({f"region_{k}": x for k, x in record["regions_pct_iod"].items()})
    buf = io.StringIO()
    pd.DataFrame([flat]).to_csv(buf, index=False)
    st.download_button("Download CSV", buf.getvalue(), "symmetry_result.csv", "text/csv")
```

experiments/common.py

```
"""Shared helpers for the experiment scripts."""
import os
import pickle
import sys

import numpy as np
import pandas as pd
from scipy.stats import pearsonr, spearmanr

sys.path.insert(0, os.path.join(os.path.dirname(__file__), "..", "src"))
from fsm import mesh as MS  # noqa: E402
from fsm.mesh import ROOT  # noqa: E402
from fsm.pipeline import asym68, measure_from_raw, signals_of, variant_outputs  # noqa: E402

TABLES = os.path.join(ROOT, "results", "tables")
FIGS = os.path.join(ROOT, "results", "figures")
RAW = os.path.join(ROOT, "results", "raw")
REGIONS = list(MS.IBUG68_REGIONS)


def load_cache(split):
    with open(os.path.join(RAW, f"detections_{split}.pkl"), "rb") as f:
        return pickle.load(f)
```

```
def reference(gt68):
    """Reference asymmetry from the human-annotated landmarks (same measure as M1)."""
    return asym68(gt68)


def row_for(raw, gt68, gate=None, extra=None):
    """
    One CSV row: reference asymmetry, every variant's output and all gate signals.
    Scores are stored in % of IOD (x100) for readability.
    """
    ref = reference(gt68)
    m = measure_from_raw(raw)
    v = variant_outputs(m, gate)
    r = {"ok": m["ok"], "failure": m.get("failure"),
         "A_ref": 100 * ref.global_score,
         **{f"ref_{k}": 100 * x for k, x in ref.region_scores.items()}}
    if extra:
        r.update(extra)
    for k in ("M0", "M1", "M2", "M3"):
        r[f"{k}_score"] = 100 * v[k]["score"] if np.isfinite(v[k]["score"]) else np.nan
        r[f"{k}_status"] = v[k]["status"]
        r[f"{k}_risk"] = v[k]["risk"]
    if m["ok"]:
        for k in REGIONS:
            r[f"M1_{k}"] = 100 * v["M1"]["regions"][k]
            if v["M3"]["regions"]:
                r[f"M3_{k}"] = 100 * v["M3"]["regions"][k]
                r[f"M3_{k}_valid"] = v["M3"]["region_valid"][k]
        s = signals_of(m, use_mirror=True)
        r.update({f"sig_{k}": val for k, val in s.items()})
        r["yaw"], r["pitch"], r["roll"] = (m["pose"][q] for q in ("yaw", "pitch", "roll"))
        G = gt68
        iod = np.linalg.norm(G[36] - G[45])
        r["nme_inner_pct"] = 100 * np.mean(np.linalg.norm(
            m["lm68"][MS.IBUG68_INNER] - G[MS.IBUG68_INNER], axis=1)) / iod
        if m.get("mirror"):
            r["M3_band"] = 100 * v["M3"]["stability_band"]
            for k in REGIONS:
                r[f"incons_{k}"] = m["mirror"]["region_incons"][k]
        b = m.get("baseline")
        if b:
            r["M0fixed_score"] = 100 * b["fixed_pair_score"]
            r["M0noalign_score"] = 100 * b["noalign_score"]
            r["M0_cross_rate"] = b["cross_structure_rate"]
        r["detect_s"] = m.get("detect_s")
    return r


def agreement(df, col, ref="A_ref"):
    d = df[[col, ref]].dropna()
    if len(d) < 3:
        return {"n": len(d), "spearman": np.nan, "pearson": np.nan, "MAE": np.nan}
    return {"n": len(d), "spearman": spearmanr(d[col], d[ref]).statistic,
            "pearson": pearsonr(d[col], d[ref]).statistic,
            "MAE": float(np.mean(np.abs(d[col] - d[ref])))}


def risk_coverage(err, risk):
    """Selective-prediction curve: mean error of the retained set when keeping the lowest-risk fraction."""
    o = np.argsort(risk, kind="stable")
    e = np.asarray(err)[o]
    n = len(e)
    cov = np.arange(1, n + 1) / n
    risk_curve = np.cumsum(e) / np.arange(1, n + 1)
    return cov, risk_curve, float(np.mean(risk_curve))   # AURC = mean over coverages
```

experiments/cache_detections.py

```
"""
Run the detector once on every LFPW image (original, mirrored, baseline-aligned) and
cache the raw outputs, so all clean-image experiments use identical detections.

    python experiments/cache_detections.py --split train
    python experiments/cache_detections.py --split test
Writes results/raw/detections_<split>.pkl
"""
import argparse
import os
import pickle
import sys

import cv2

sys.path.insert(0, os.path.join(os.path.dirname(__file__), "..", "src"))
from fsm.data import crop_to_annotation, list_split, read_pts  # noqa: E402
from fsm.pipeline import raw_detect  # noqa: E402
from fsm.mesh import ROOT  # noqa: E402

ap = argparse.ArgumentParser()
ap.add_argument("--split", required=True, choices=["train", "test"])
```

```
args = ap.parse_args()

rows = []
items = list_split(args.split)
for k, (im, pts) in enumerate(items):
    img, gt = crop_to_annotation(cv2.imread(im), read_pts(pts))
    raw = raw_detect(img)
    raw["image"] = os.path.basename(im)
    raw["gt68"] = gt
    rows.append(raw)
    if k % 100 == 0:
        print(f"{args.split}: {k}/{len(items)}", flush=True)

out = os.path.join(ROOT, "results", "raw", f"detections_{args.split}.pkl")
with open(out, "wb") as f:
    pickle.dump(rows, f)
n_ok = sum(r["ok"] for r in rows)
print(f"saved {out}: {n_ok}/{len(rows)} faces detected")
```

experiments/fit_mp68_mapping.py

```
"""
Fit the MediaPipe-vertex <-> iBUG-68 correspondence on the LFPW TRAINING split.

For every iBUG point k and mesh vertex v we compute the median (over training images)
of ||L_v - GT_k|| / IOD_gt. The mapping is chosen under a mirror constraint so that the
correspondence itself cannot create left-right asymmetry:
  * bilateral pair (a, b):  pick v minimising E[a, v] + E[b, mirror(v)]; a->v, b->mirror(v)
  * midline point k:        pick v among midline vertices (mirror(v) == v)
The fitted mapping is then evaluated on the TEST split (NME, inner 51 points) and
compared with the commonly circulated hand-made mapping.

Writes configs/mp68_mapping.json and results/tables/mapping_nme.csv
"""
import json
import os
import pickle
import sys

import numpy as np
import pandas as pd

sys.path.insert(0, os.path.join(os.path.dirname(__file__), "..", "src"))
from fsm import mesh as MS  # noqa: E402
from fsm.mesh import ROOT  # noqa: E402


def load(split):
    with open(os.path.join(ROOT, "results", "raw", f"detections_{split}.pkl"), "rb") as f:
        return [r for r in pickle.load(f) if r["ok"]]


def nme(mapping, rows, idx):
    """Per-image NME (inter-ocular normalised mean error) on point subset idx."""
    out = []
    for r in rows:
        P, G = r["L"][mapping], r["gt68"]
        d = np.linalg.norm(G[36] - G[45])
        out.append(np.mean(np.linalg.norm(P[idx] - G[idx], axis=1)) / d)
    return np.array(out)


train, test = load("train"), load("test")
perm = MS.mp_permutation()
# E[k, v]: median normalised error of vertex v as a stand-in for iBUG point k
errs = []
for r in train:
    G = r["gt68"]
    d = np.linalg.norm(G[36] - G[45])
    L = r["L"][:468]
    errs.append(np.linalg.norm(G[:, None, :] - L[None, :, :], axis=2) / d)
E = np.median(np.stack(errs), axis=0)                       # (68, 468)

mapping = np.zeros(68, dtype=int)
central_v = np.where(perm[:468] == np.arange(468))[0]
for a, b in MS.IBUG68_PAIRS:
    cost = E[a] + E[b, perm[:468]]
    v = int(np.argmin(cost))
    mapping[a], mapping[b] = v, int(perm[v])
for k in MS.IBUG68_CENTRAL:
    mapping[k] = int(central_v[np.argmin(E[k, central_v])])

# Sanity: the mapping must commute with mirroring.
P68 = MS.ibug68_permutation()
assert np.all(perm[mapping] == mapping[P68]), "mapping is not mirror-consistent"

hand = np.asarray(MS.HANDMADE_MP68)
rows = []
for name, m in [("hand-made", hand), ("fitted (train)", mapping)]:
    for subset, idx in [("inner51", MS.IBUG68_INNER), ("jaw17", list(range(17))),
```

```
                        ("all68", list(range(68)))]:
        e = nme(m, test, idx)
        rows.append({"mapping": name, "points": subset, "test_NME_mean_pct": 100 * e.mean(),
                     "test_NME_median_pct": 100 * np.median(e), "n_images": len(e)})
df = pd.DataFrame(rows)
df.to_csv(os.path.join(ROOT, "results", "tables", "mapping_nme.csv"), index=False)
print(df.round(3).to_string(index=False))
hand_sym = bool(np.all(perm[hand] == hand[P68]))
print("hand-made mapping mirror-consistent:", hand_sym)

with open(os.path.join(ROOT, "configs", "mp68_mapping.json"), "w") as f:
    json.dump({"mp_index_for_ibug68": mapping.tolist(),
               "fitted_on": "LFPW trainset (%d images)" % len(train),
               "handmade_mirror_consistent": hand_sym}, f, indent=1)
```

experiments/calibrate_gate.py

```
"""
Calibrate the data-dependent gate thresholds on the LFPW TRAINING split only.

Rules (written down before looking at test results):
  * a-priori limits from the measurement's assumptions (not tuned):
      |yaw|   caution 10 deg, reject 20 deg   (left-right comparison in 2D)
      |pitch| caution 15 deg, reject 25 deg
      IOD     caution 60 px,  reject 30 px    (landmark quantisation becomes ~1-3 % of IOD)
      exposure reject: mean luma < 35 or > 225
  * statistic limits from the distribution of *clean* training images:
      sharpness caution = 2.5th percentile, reject = 0.5th percentile of train - 0.3
      brightness caution = 2.5th / 97.5th percentiles
      clipped caution = 97.5th percentile, reject = max(0.6, caution + 0.15)
      mirror inconsistency caution = 90th percentile, reject = 99th percentile
      per-region mirror limit = 97.5th percentile over all regions
  * readable index S = 100 / (1 + alpha * A[%]) with alpha s.t. the train median maps to 50.
Writes configs/gate_thresholds.json
"""
import json
import os

import numpy as np
import pandas as pd

from common import REGIONS, TABLES, load_cache, row_for
from fsm.mesh import CONFIG_DIR
from fsm.quality import DEFAULT_GATE

rows = [row_for(r, r["gt68"], gate=DEFAULT_GATE, extra={"image": r["image"]})
        for r in load_cache("train")]
df = pd.DataFrame(rows)
df.to_csv(os.path.join(TABLES, "train_clean_rows.csv"), index=False)
ok = df[df.ok]
q = lambda c, p: float(np.percentile(ok[c].dropna(), p))  # noqa: E731

g = {k: dict(v) for k, v in DEFAULT_GATE.items()}
g["sharpness"].update(caution=q("sig_sharpness", 2.5), reject=q("sig_sharpness", 0.5) - 0.3)
g["brightness_lo"]["caution"] = q("sig_brightness", 2.5)
g["brightness_hi"]["caution"] = q("sig_brightness", 97.5)
g["clipped"]["caution"] = q("sig_clipped", 97.5)
# the face crop includes some background, so clipping is common; keep reject above caution
g["clipped"]["reject"] = max(0.6, g["clipped"]["caution"] + 0.15)
g["mirror_incons"].update(caution=q("sig_mirror_incons", 90), reject=q("sig_mirror_incons", 99))
region_limit = float(np.percentile(np.concatenate(
    [ok[f"incons_{k}"].dropna().values for k in REGIONS]), 97.5))
alpha = 1.0 / float(np.median(ok["M3_score"].dropna()))     # A in % of IOD

cfg = {"thresholds": g, "region_mirror_limit": region_limit, "index_alpha_per_pct": alpha,
       "calibrated_on": f"LFPW trainset, {len(ok)} detected faces of {len(df)}",
       "rules": __doc__}
with open(os.path.join(CONFIG_DIR, "gate_thresholds.json"), "w") as f:
    json.dump(cfg, f, indent=1)
print(json.dumps({k: v for k, v in cfg.items() if k != "rules"}, indent=1))
```

experiments/run_clean_eval.py

```
"""
Clean-image evaluation on the LFPW TEST split (experiments E1, E2, E3, E8 and the
clean part of E9).

Reference: the same asymmetry measure computed on the human-annotated 68 landmarks
(A_ref). This is NOT a human symmetry rating; it is the geometric quantity we want to
measure, computed from expert landmarks instead of detector landmarks. Measurement
error = |A_detector - A_ref|.

Outputs (results/tables): test_clean_rows.csv, agreement_global.csv, agreement_regions.csv,
correspondence_ablation.csv, risk_coverage.csv, gate_operating_point.csv,
pose_strata.csv
"""
import os
```

```python
import numpy as np
import pandas as pd
from scipy.stats import spearmanr

from common import REGIONS, TABLES, agreement, load_cache, risk_coverage, row_for

df = pd.DataFrame([row_for(r, r["gt68"], extra={"image": r["image"]})
                   for r in load_cache("test")])
df.to_csv(os.path.join(TABLES, "test_clean_rows.csv"), index=False)
ok = df[df.ok].copy()
print(f"test images: {len(df)}, detected: {len(ok)}")

# ---------------- E1/E2: agreement of each measure with the reference ----------------
rows = []
for name, col in [("M0 baseline (KD-tree, 478 mesh, align+redetect)", "M0_score"),
                  ("M0 without alignment", "M0noalign_score"),
                  ("M1 fixed pairs + fitted axis (68)", "M1_score"),
                  ("M3 mirror-averaged landmarks (68)", "M3_score")]:
    a = agreement(ok, col)
    # MAE only meaningful for measures on the same scale as A_ref (68-layout, % IOD)
    if not col.startswith("M1") and not col.startswith("M3"):
        a["MAE"] = np.nan
    rows.append({"measure": name, **a})
ag = pd.DataFrame(rows)
ag.to_csv(os.path.join(TABLES, "agreement_global.csv"), index=False)
print(ag.round(3).to_string(index=False))

# ---------------- E3: regional agreement ----------------------------------------------
rows = []
for k in REGIONS:
    for v in ("M1", "M3"):
        a = agreement(ok, f"{v}_{k}", ref=f"ref_{k}")
        rows.append({"region": k, "variant": v, **a,
                     "ref_mean": ok[f"ref_{k}"].mean(), "det_mean": ok[f"{v}_{k}"].mean()})
rg = pd.DataFrame(rows)
rg.to_csv(os.path.join(TABLES, "agreement_regions.csv"), index=False)
print(rg.round(3).to_string(index=False))

# ---------------- correspondence ablation (same detections, same midline) ------------
cr = pd.DataFrame([
    {"correspondence": "KD-tree reflected nearest neighbour", **agreement(ok, "M0_score")},
    {"correspondence": "fixed canonical mirror pairs", **agreement(ok, "M0fixed_score")},
])
cr["MAE"] = np.nan
cr["kdtree_wrong_partner_rate_mean"] = [ok["M0_cross_rate"].mean(), 0.0]
cr.to_csv(os.path.join(TABLES, "correspondence_ablation.csv"), index=False)
print(cr.round(3).to_string(index=False))

# ---------------- E9 (clean): gate as a selective-measurement mechanism ---------------
ok["err_M1"] = (ok["M1_score"] - ok["A_ref"]).abs()
ok["err_M3"] = (ok["M3_score"] - ok["A_ref"]).abs()
rng = np.random.default_rng(0)
curves, summ = [], []
signals = {
    "random": (rng.random(len(ok)), "err_M1"),
    "M2 gate risk (size/blur/exposure/pose)": (ok["M2_risk"].values, "err_M1"),
    "|yaw| only": (ok["sig_abs_yaw"].values, "err_M1"),
    "mirror inconsistency only": (ok["sig_mirror_incons"].values, "err_M1"),
    "M3 gate risk (M2 + mirror), M3 score": (ok["M3_risk"].values, "err_M3"),
}
for name, (risk, ecol) in signals.items():
    cov, rc, aurc = risk_coverage(ok[ecol].values, risk)
    curves.append(pd.DataFrame({"signal": name, "coverage": cov, "mean_abs_err": rc}))
    summ.append({"signal": name, "error_col": ecol, "AURC": aurc,
                 "err_at_100pct": rc[-1], "err_at_80pct": rc[int(0.8 * len(rc)) - 1],
                 "err_at_50pct": rc[int(0.5 * len(rc)) - 1],
                 "spearman_risk_vs_err": spearmanr(risk, ok[ecol]).statistic})
pd.concat(curves).to_csv(os.path.join(TABLES, "risk_coverage.csv"), index=False)
sm = pd.DataFrame(summ)
sm.to_csv(os.path.join(TABLES, "risk_coverage_summary.csv"), index=False)
print(sm.round(4).to_string(index=False))

# Operating point of the discrete gate
op = []
for v, ecol in (("M2", "err_M1"), ("M3", "err_M3")):
    for st in ("accept", "caution", "reject"):
        s = ok[ok[f"{v}_status"] == st]
        op.append({"variant": v, "status": st, "n": len(s), "pct": 100 * len(s) / len(ok),
                   "mean_abs_err": s[ecol].mean() if len(s) else np.nan,
                   "median_abs_err": s[ecol].median() if len(s) else np.nan,
                   "mean_nme_inner": s["nme_inner_pct"].mean() if len(s) else np.nan})
op = pd.DataFrame(op)
op.to_csv(os.path.join(TABLES, "gate_operating_point.csv"), index=False)
print(op.round(3).to_string(index=False))

# Which signal triggered each non-accept decision (M3 gate)
from fsm.quality import gate_decision, load_gate  # noqa: E402
gate = load_gate()
cnt = {}
for _, r in ok.iterrows():
    sig = {k[4:]: r[k] for k in ok.columns if k.startswith("sig_")}
```

```
    for why in gate_decision(sig, gate, use_mirror=True)[1]:
        key = (why["signal"], why["level"])
        cnt[key] = cnt.get(key, 0) + 1
rc = pd.DataFrame([{"signal": k[0], "level": k[1], "n_images": v} for k, v in cnt.items()])
rc = rc.sort_values(["level", "n_images"], ascending=[False, False])
rc.to_csv(os.path.join(TABLES, "gate_reasons.csv"), index=False)
print(rc.to_string(index=False))

# Landmark error (NME) as the target: can the mirror test detect bad landmarks
# without ground truth?
rows = []
for name, risk in [("random", rng.random(len(ok))), ("M2 gate risk", ok["M2_risk"].values),
                   ("mirror inconsistency", ok["sig_mirror_incons"].values),
                   ("M3 gate risk", ok["M3_risk"].values),
                   ("oracle (true NME)", ok["nme_inner_pct"].values)]:
    cov, rcurve, aurc = risk_coverage(ok["nme_inner_pct"].values, risk)
    rows.append({"signal": name, "AURC_nme": aurc, "nme_at_80pct": rcurve[int(0.8 * len(rcurve)) - 1],
                 "nme_at_50pct": rcurve[int(0.5 * len(rcurve)) - 1],
                 "spearman_vs_nme": spearmanr(risk, ok["nme_inner_pct"]).statistic})
    curves.append(pd.DataFrame({"signal": "NME|" + name, "coverage": cov, "mean_abs_err": rcurve}))
nm = pd.DataFrame(rows)
nm.to_csv(os.path.join(TABLES, "risk_coverage_nme.csv"), index=False)
pd.concat(curves).to_csv(os.path.join(TABLES, "risk_coverage.csv"), index=False)
print(nm.round(4).to_string(index=False))

# Pose confound: how strongly does each score follow |yaw|, before and after gating?
pc = []
for subset_name, sub in [("all", ok), ("M3 accept", ok[ok.M3_status == "accept"]),
                         ("M3 accept+caution", ok[ok.M3_status != "reject"])]:
    pc.append({"subset": subset_name, "n": len(sub),
               "rho(A_ref,|yaw|)": spearmanr(sub["A_ref"], sub["sig_abs_yaw"]).statistic,
               "rho(M3,|yaw|)": spearmanr(sub["M3_score"], sub["sig_abs_yaw"]).statistic,
               "rho(M0,|yaw|)": spearmanr(sub["M0_score"], sub["sig_abs_yaw"]).statistic,
               "A_ref_mean": sub["A_ref"].mean(), "M3_mean": sub["M3_score"].mean()})
pc = pd.DataFrame(pc)
pc.to_csv(os.path.join(TABLES, "pose_confound.csv"), index=False)
print(pc.round(3).to_string(index=False))

# Shrinkage: detector-based asymmetry vs annotation-based (paired, per region)
sh = []
from scipy.stats import wilcoxon  # noqa: E402
for k in ["global"] + REGIONS:
    a = ok["A_ref"] if k == "global" else ok[f"ref_{k}"]
    b = ok["M3_score"] if k == "global" else ok[f"M3_{k}"]
    sh.append({"region": k, "ref_mean": a.mean(), "detector_mean": b.mean(),
               "ratio": b.mean() / a.mean(), "wilcoxon_p": wilcoxon(a, b).pvalue})
sh = pd.DataFrame(sh)
sh.to_csv(os.path.join(TABLES, "shrinkage.csv"), index=False)
print(sh.round(4).to_string(index=False))

# ---------------- E8: pose stratification ------------------------------------------
ok["yaw_bin"] = pd.cut(ok["sig_abs_yaw"], [0, 5, 10, 20, 90], right=False,
                       labels=["0-5", "5-10", "10-20", ">=20"])
ps = ok.groupby("yaw_bin", observed=False).agg(
    n=("A_ref", "size"), A_ref_mean=("A_ref", "mean"), M1_mean=("M1_score", "mean"),
    err_M1=("err_M1", "mean"), err_M3=("err_M3", "mean"),
    nme_inner=("nme_inner_pct", "mean"), mirror_incons=("sig_mirror_incons", "mean"),
    M0_mean=("M0_score", "mean")).reset_index()
ps.to_csv(os.path.join(TABLES, "pose_strata.csv"), index=False)
print(ps.round(3).to_string(index=False))
print("corr(mirror_incons, nme):", spearmanr(ok["sig_mirror_incons"], ok["nme_inner_pct"]).statistic)
print("corr(mirror_incons, err_M1):", spearmanr(ok["sig_mirror_incons"], ok["err_M1"]).statistic)
print("mean runtime per image (3 detections) s:", ok["detect_s"].mean())
```

experiments/run_perturbations.py

```
"""
Controlled perturbation study on the LFPW TEST split (experiments E4-E7 + face size).

Every test image (cropped to the annotated face, exactly as in the clean evaluation) is
perturbed with fixed, deterministic settings; the full pipeline is re-run, and the
annotation is transformed with the image where geometry changes (rotation, scaling).
Since the true face geometry does not change, the reference asymmetry A_ref is
unchanged (it is invariant to rotation and scale by construction).

Settings (no randomness; occluder positions come from the annotation):
  brightness  pixel values x {0.25, 0.4, 0.6, 1.5, 2.0, 2.5}  (clipped to [0, 255])
  blur        Gaussian, sigma = s * IOD / 100,  s in {1, 2, 4, 6, 8}
  occlusion   mid-grey box over one eye / the nose / the mouth (annotation box + 35 %)
  rotation    in-plane, {-30, -20, -10, -5, 5, 10, 20, 30} deg, canvas enlarged (no crop)
  downscale   image resized by {0.5, 0.3, 0.2, 0.12}
Writes results/tables/perturbation_rows.csv
"""
import os
import sys

import cv2
import numpy as np
import pandas as pd
```

```
from common import TABLES, load_cache, row_for
from fsm.data import crop_to_annotation, list_split, read_pts
from fsm.pipeline import raw_detect


def rotate(img, gt, angle):
    h, w = img.shape[:2]
    M = cv2.getRotationMatrix2D((w / 2, h / 2), angle, 1.0)
    c, s = abs(M[0, 0]), abs(M[0, 1])
    nw, nh = int(h * s + w * c), int(h * c + w * s)
    M[0, 2] += nw / 2 - w / 2
    M[1, 2] += nh / 2 - h / 2
    out = cv2.warpAffine(img, M, (nw, nh), borderMode=cv2.BORDER_REPLICATE)
    return out, gt @ M[:, :2].T + M[:, 2]


def occlude(img, gt, idx, pad=0.35):
    out = img.copy()
    x0, y0 = gt[idx].min(0)
    x1, y1 = gt[idx].max(0)
    px, py = pad * (x1 - x0) + 4, pad * (y1 - y0) + 4
    cv2.rectangle(out, (int(x0 - px), int(y0 - py)), (int(x1 + px), int(y1 + py)),
                  (128, 128, 128), -1)
    return out, gt


PERTURBATIONS = (
    [("brightness", k, lambda im, g, k=k: (np.clip(im.astype(np.float32) * k, 0, 255)
                                           .astype(np.uint8), g))
     for k in (0.25, 0.4, 0.6, 1.5, 2.0, 2.5)]
    + [("blur", s, lambda im, g, s=s: (cv2.GaussianBlur(
        im, (0, 0), s * np.linalg.norm(g[36] - g[45]) / 100.0), g))
       for s in (1, 2, 4, 6, 8)]
    + [("occlusion", name, lambda im, g, idx=idx: occlude(im, g, idx))
       for name, idx in (("one eye", list(range(36, 42))), ("nose", list(range(27, 36))),
                         ("mouth", list(range(48, 68))))]
    + [("rotation", a, lambda im, g, a=a: rotate(im, g, a))
       for a in (-30, -20, -10, -5, 5, 10, 20, 30)]
    + [("downscale", f, lambda im, g, f=f: (cv2.resize(im, None, fx=f, fy=f,
                                                       interpolation=cv2.INTER_AREA), g * f))
       for f in (0.5, 0.3, 0.2, 0.12)]
)

if __name__ == "__main__":
    limit = int(sys.argv[1]) if len(sys.argv) > 1 else None
    items = list_split("test")[:limit]
    rows = []
    for n, (im, pts) in enumerate(items):
        img, gt = crop_to_annotation(cv2.imread(im), read_pts(pts))
        name = os.path.basename(im)
        for kind, level, fn in PERTURBATIONS:
            pimg, pgt = fn(img, gt)
            r = row_for(raw_detect(pimg), pgt, extra={"image": name, "perturbation": kind,
                                                      "level": str(level)})
            rows.append(r)
        if n % 20 == 0:
            print(f"{n}/{len(items)}", flush=True)
    df = pd.DataFrame(rows)
    df.to_csv(os.path.join(TABLES, "perturbation_rows.csv"), index=False)
    print("saved", len(df), "rows")
```

experiments/summarize_perturbations.py

```
"""
Summarise the perturbation study (E4-E7) and build the ablation table (E9).

Metrics (all defined before running; tau fixed a priori at 1.0 % IOD, about twice the
clean-image mean absolute error):
  detect_rate       fraction of perturbed images where a face is found
  drift_<M>_sd      median |score(perturbed) - score(clean)| / SD of clean scores of M
                    (scale-free, so M0 (face-width units) and M1/M3 (IOD units) compare)
  mae_<M>           mean |A_det - A_ref| over images where a score is produced
  flag_<M>          fraction not accepted (caution or reject; failures count as reject)
  reject_<M>        fraction rejected
  silent_<M>        fraction of images where a score is REPORTED (status != reject) and
                    its error exceeds tau  -> the failure mode the gate must prevent
Writes results/tables/perturbation_summary.csv and ablation.csv
"""
import os

import numpy as np
import pandas as pd
from scipy.stats import spearmanr

from common import TABLES

TAU = 1.0
clean = pd.read_csv(os.path.join(TABLES, "test_clean_rows.csv"))
pert = pd.read_csv(os.path.join(TABLES, "perturbation_rows.csv"))
sd = {k: clean[f"{k}_score"].std() for k in ("M0", "M0noalign", "M1", "M3")}
```

```
base = clean.set_index("image")
for k in ("M0", "M0noalign", "M1", "M3"):
    pert[f"clean_{k}"] = pert["image"].map(base[f"{k}_score"])
    pert[f"drift_{k}"] = (pert[f"{k}_score"] - pert[f"clean_{k}"]).abs() / sd[k]
for k in ("M1", "M3"):
    pert[f"err_{k}"] = (pert[f"{k}_score"] - pert["A_ref"]).abs()
for k in ("M2", "M3"):
    pert[f"{k}_status"] = pert[f"{k}_status"].fillna("reject")


def summarise(d):
    r = {"n": len(d), "detect_rate": d["ok"].mean()}
    for k in ("M0", "M0noalign", "M1", "M3"):
        r[f"drift_{k}_sd"] = d[f"drift_{k}"].median()
    r["mae_M1"] = d["err_M1"].mean()
    r["mae_M3"] = d["err_M3"].mean()
    for k in ("M2", "M3"):
        r[f"flag_{k}"] = (d[f"{k}_status"] != "accept").mean()
        r[f"reject_{k}"] = (d[f"{k}_status"] == "reject").mean()
    r["silent_M1"] = ((d["err_M1"] > TAU) & d["ok"]).mean()
    r["silent_M2"] = ((d["err_M1"] > TAU) & (d["M2_status"] != "reject")).mean()
    r["silent_M3"] = ((d["err_M3"] > TAU) & (d["M3_status"] != "reject")).mean()
    return pd.Series(r)


summ = pert.groupby(["perturbation", "level"], sort=False).apply(
    summarise, include_groups=False).reset_index()
summ.to_csv(os.path.join(TABLES, "perturbation_summary.csv"), index=False)
pd.set_option("display.width", 250)
print(summ.round(3).to_string(index=False))

# ---------------------------------------------------------------- E9 ablation
clean = clean.copy()
clean["err_M1"] = (clean["M1_score"] - clean["A_ref"]).abs()
clean["err_M3"] = (clean["M3_score"] - clean["A_ref"]).abs()


def ablation_rows(d, setting):
    rows = []
    spec = [("M0", "M0", None, "baseline: KD-tree, vertical midline, align+redetect"),
            ("M1", "M1", None, "+ fixed pairs, fitted axis, 68-pt, IOD"),
            ("M2", "M1", "M2_status", "+ quality/pose gate"),
            ("M3", "M3", "M3_status", "+ mirror-averaged landmarks, mirror signal, region validity")]
    for name, scol, stcol, desc in spec:
        reported = d["ok"] if stcol is None else (d[stcol].fillna("reject") != "reject")
        accepted = d["ok"] if stcol is None else (d[stcol].fillna("reject") == "accept")
        sc = d[f"{scol}_score"]
        r = {"setting": setting, "variant": name, "description": desc, "n": len(d),
             "coverage_reported": reported.mean(), "coverage_accepted": accepted.mean()}
        dd = d[reported & sc.notna()]
        r["spearman_vs_ref"] = spearmanr(dd[f"{scol}_score"], dd["A_ref"]).statistic if len(dd) > 2 else np.nan
        r["MAE_reported"] = dd[f"err_{scol}"].mean() if scol != "M0" else np.nan
        r["silent_failure_rate"] = (((d[f"err_{scol}"] > TAU) & reported).mean()
                                    if scol != "M0" else np.nan)
        if "drift_M0" in d:
            r["median_drift_sd"] = d[f"drift_{scol}"].median()
        rows.append(r)
    return rows


rows = ablation_rows(clean, "clean test (224)")
for kind in ("brightness", "blur", "occlusion", "rotation", "downscale"):
    rows += ablation_rows(pert[pert.perturbation == kind], f"perturbed: {kind}")
rows += ablation_rows(pert, "perturbed: all (pooled)")
ab = pd.DataFrame(rows)
ab.to_csv(os.path.join(TABLES, "ablation.csv"), index=False)
print(ab.drop(columns="description").round(3).to_string(index=False))

# Region validity under occlusion: is the occluded region the one marked unreliable?
occ = pert[(pert.perturbation == "occlusion") & pert.ok]
target = {"one eye": "Eyes", "nose": "Nose", "mouth": "Mouth"}
rv = []
for lvl, reg in target.items():
    d = occ[occ.level == lvl]
    cl = clean.set_index("image")
    rv.append({"occluded": lvl, "n": len(d),
               "occluded_region_flagged": (~d[f"M3_{reg}_valid"].astype(bool)).mean(),
               "other_regions_flagged": np.mean([(~d[f"M3_{o}_valid"].astype(bool)).mean()
                                                 for o in target.values() if o != reg]),
               "clean_flag_rate_same_region": (~cl[f"M3_{reg}_valid"].astype(bool)).mean(),
               "region_err_M1": (d[f"M1_{reg}"] - d[f"ref_{reg}"]).abs().mean(),
               "region_err_clean": (cl[f"M1_{reg}"] - cl[f"ref_{reg}"]).abs().mean()})
rv = pd.DataFrame(rv)
rv.to_csv(os.path.join(TABLES, "region_validity_occlusion.csv"), index=False)
print(rv.round(3).to_string(index=False))
```

experiments/make_figures.py

```
"""
Generate every paper figure from the saved CSVs / cached detections (no manual edits).
```

```
    python experiments/make_figures.py
Writes results/figures/*.pdf and *.png
"""
import os

import cv2
import matplotlib
import numpy as np
import pandas as pd

matplotlib.use("Agg")
import matplotlib.pyplot as plt  # noqa: E402

from common import FIGS, TABLES, load_cache  # noqa: E402
from fsm import mesh as MS, viz  # noqa: E402
from fsm.data import crop_to_annotation, list_split, read_pts  # noqa: E402
from fsm.pipeline import PERM478, measure_from_raw, variant_outputs, raw_detect  # noqa: E402

# Validated categorical slots (dataviz reference palette, light mode) + status palette
C = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#008300", "#6250d6", "#e34948"]
STATUS = {"accept": "#0ca30c", "caution": "#fab219", "reject": "#d03b3b"}
STATUS_MARK = {"accept": "o", "caution": "^", "reject": "X"}
INK, MUTED, GRID = "#1f1f1e", "#6b6a64", "#e4e3dd"
plt.rcParams.update({
    "font.family": "DejaVu Sans", "font.size": 8, "axes.titlesize": 8.5,
    "axes.labelsize": 8, "axes.edgecolor": MUTED, "axes.labelcolor": INK,
    "xtick.color": MUTED, "ytick.color": MUTED, "axes.grid": True, "grid.color": GRID,
    "grid.linewidth": 0.6, "axes.spines.top": False, "axes.spines.right": False,
    "legend.frameon": False, "lines.linewidth": 2, "savefig.dpi": int(os.environ.get("FIG_DPI", 220)),
})


def save(fig, name):
    fig.savefig(os.path.join(FIGS, name + ".pdf"), bbox_inches="tight")
    fig.savefig(os.path.join(FIGS, name + ".png"), bbox_inches="tight")
    plt.close(fig)
    print("wrote", name)


def rgb(img):
    return cv2.cvtColor(img, cv2.COLOR_BGR2RGB)


test_rows = pd.read_csv(os.path.join(TABLES, "test_clean_rows.csv"))
cache = {r["image"]: r for r in load_cache("test")}
ok = test_rows[test_rows.ok].copy()


def canvas(img):
    """
    Privacy rule for figures: dataset photographs (which include identifiable people,
    some of them public figures) are never reproduced in the paper. Geometry is drawn on
    a blank canvas of the same size instead.
    """
    return np.full_like(img, 248)


DARK = (60, 60, 60)
BLUE = (214, 120, 42)    # C[0] in BGR


def load_test_image(name):
    d = dict(list_split("test"))
    path = [p for p in d if os.path.basename(p) == name][0]
    return crop_to_annotation(cv2.imread(path), read_pts(d[path]))


# ---------------------------------------------------------------------- Fig: pipeline strip
def fig_pipeline(name):
    img, gt = load_test_image(name)
    m = measure_from_raw(cache[name])
    v = variant_outputs(m)
    P = m["mirror"]["lm68_avg"]
    res = m["mirror"]["avg"]
    bg = canvas(img)
    U = m["mirror"]["lm478_unflipped"][:468]
    both = viz.draw_mesh(viz.draw_mesh(bg, U, BLUE, radius=3), m["lm478"][:468], DARK, radius=1)
    panels = [
        ("(a) 478-pt mesh on I", viz.draw_mesh(bg, m["lm478"][:468], DARK)),
        ("(b) I + unflip(flip(I))", both),
        ("(c) mapped 68 points", viz.draw_regions(bg, P)),
        ("(d) fitted axis + pairs", viz.draw_axis(viz.draw_pairs(bg, P, color=BLUE), res, P)),
        ("(e) mirror residuals", viz.draw_mirrored(bg, res, P)),
        ("(f) per-point heatmap", viz.draw_heatmap(bg, res, P, valid=v["M3"]["region_valid"])),
    ]
    fig, ax = plt.subplots(1, 6, figsize=(7.0, 1.6))
    for a, (t, im) in zip(ax, panels):
        a.imshow(rgb(im))
        a.set_title(t, fontsize=7)
        a.axis("off")
    fig.tight_layout(pad=0.2)
```

```
    save(fig, "pipeline_examples")
    return v


# ---------------------------------------------------------------------- Fig: KD-tree vs fixed
def fig_correspondence(name):
    img, _ = load_test_image(name)
    m = measure_from_raw(cache[name])
    b = m["baseline"]
    La = b["lm478_aligned"]
    aligned = cv2.warpAffine(canvas(img), cv2.estimateAffinePartial2D(
        np.float32([m["lm478"][33], m["lm478"][263]]),
        np.float32([[600 * .33, 600 * .33], [600 * .67, 600 * .33]]))[0], (600, 600))
    # Reflected left points vs their KD-tree partners (draw left point -> partner)
    kd = viz.draw_kdtree_matches(aligned, La, b["pairs"], PERM478)
    fixed_pairs = np.stack([b["pairs"][:, 0], PERM478[b["pairs"][:, 0]]], 1)
    fx = viz.draw_kdtree_matches(aligned, La, fixed_pairs, PERM478)
    fig, ax = plt.subplots(1, 2, figsize=(3.4, 1.9))
    for a, im, t in zip(ax, [kd, fx], [f"KD-tree matching\n{100*b['cross_structure_rate']:.0f}% wrong partner (red)",
                                       "fixed canonical pairs\n0% wrong partner"]):
        a.imshow(rgb(im[60:560, 60:540]))
        a.set_title(t, fontsize=7)
        a.axis("off")
    fig.tight_layout(pad=0.2)
    save(fig, "correspondence")


# ---------------------------------------------------------------------- Fig: agreement scatter
def fig_agreement():
    ag = pd.read_csv(os.path.join(TABLES, "agreement_global.csv"))
    fig, ax = plt.subplots(1, 2, figsize=(3.4, 1.75))
    for a, col, lab, c in [(ax[0], "M0_score", "M0 baseline score (% face width)", C[1]),
                           (ax[1], "M3_score", "M3 score (% IOD)", C[0])]:
        a.scatter(ok["A_ref"], ok[col], s=9, color=c, edgecolor="white", linewidth=0.4)
        rho = ag.loc[ag.measure.str.startswith(col[:2]) & ~ag.measure.str.contains("without"),
                     "spearman"].iloc[0]
        a.set_title(f"Spearman rho = {rho:.2f}", fontsize=7.5)
        a.set_xlabel("A$_{ref}$ (% IOD)")
        a.set_ylabel(lab, fontsize=7)
        if col == "M3_score":
            lim = [0, ok[["A_ref", col]].max().max() * 1.05]
            a.plot(lim, lim, color=MUTED, lw=1, ls="--")
            a.text(lim[1] * 0.97, lim[1] * 0.05, "dashed: y = x", color=MUTED, ha="right", fontsize=6.5)
    fig.tight_layout(pad=0.3)
    save(fig, "agreement")


# ---------------------------------------------------------------------- Fig: pose confound
def fig_pose():
    fig, ax = plt.subplots(figsize=(3.3, 1.9))
    for st in ("accept", "caution", "reject"):
        s = ok[ok.M3_status == st]
        ax.scatter(s["sig_abs_yaw"], s["A_ref"], s=12, marker=STATUS_MARK[st], color=STATUS[st],
                   edgecolor="white", linewidth=0.4, label=f"{st} (n={len(s)})")
    for x in (10, 20):
        ax.axvline(x, color=MUTED, lw=1, ls=":")
    ax.set_xlabel("|yaw| estimated by MediaPipe (deg)")
    ax.set_ylabel("A$_{ref}$ from annotations (% IOD)")
    ax.legend(loc="upper left", fontsize=6.5, title="M3 gate decision", title_fontsize=6.5)
    save(fig, "pose_confound")


# ---------------------------------------------------------------------- Fig: risk-coverage
def fig_risk_coverage():
    rc = pd.read_csv(os.path.join(TABLES, "risk_coverage.csv"))
    fig, ax = plt.subplots(1, 2, figsize=(3.4, 1.8))
    sets = [("NME", [("NME|random", "random", C[7]), ("NME|M2 gate risk", "M2 gate", C[1]),
                     ("NME|mirror inconsistency", "mirror test", C[0]),
                     ("NME|oracle (true NME)", "oracle", MUTED)],
             "landmark NME (% IOD)"),
            ("asym", [("random", "random", C[7]), ("M2 gate risk (size/blur/exposure/pose)", "M2 gate", C[1]),
                      ("mirror inconsistency only", "mirror test", C[0])],
             "|A$_{det}$ - A$_{ref}$| (% IOD)")]
    for a, (_, curves, ylab) in zip(ax, sets):
        for key, lab, c in curves:
            s = rc[rc.signal == key]
            s = s[s.coverage >= 0.2]
            a.plot(100 * s.coverage, s.mean_abs_err, color=c, label=lab,
                   ls="--" if lab == "oracle" else "-", lw=1.6)
        a.set_xlabel("coverage (% images kept)")
        a.set_ylabel("mean " + ylab, fontsize=7)
    ax[0].legend(fontsize=6, loc="upper left")
    fig.tight_layout(pad=0.3)
    save(fig, "risk_coverage")


# ---------------------------------------------------------------------- Fig: robustness
def fig_robustness():
    p = os.path.join(TABLES, "perturbation_summary.csv")
    if not os.path.exists(p):
        print("skip robustness (run summarize_perturbations.py first)")
```

```
        return
    s = pd.read_csv(p)
    kinds = ["brightness", "blur", "rotation", "downscale", "occlusion"]
    xl = {"brightness": "intensity factor", "blur": "sigma (% IOD)", "rotation": "angle (deg)",
          "downscale": "scale factor", "occlusion": "occluded part"}
    fig, ax = plt.subplots(2, 5, figsize=(7.0, 3.0))
    for j, k in enumerate(kinds):
        d = s[s.perturbation == k].copy()
        if k == "occlusion":
            x = np.arange(len(d))
        else:
            d["lv"] = d.level.astype(float)
            d = d.sort_values("lv")
            x = d.lv.values
        for col, lab, c in [("drift_M0_sd", "M0 baseline", C[1]), ("drift_M1_sd", "M1 fixed+axis", C[2]),
                            ("drift_M3_sd", "M3 mirror-avg", C[0])]:
            ax[0, j].plot(x, d[col], marker="o", ms=3, color=c, label=lab, lw=1.5)
        if k == "rotation":
            ax[0, j].plot(x, d["drift_M0noalign_sd"], marker="s", ms=3, color=C[1], ls="--",
                          lw=1.2, label="M0 w/o alignment")
            ax[0, j].set_yscale("log")
        for col, lab, c in [("flag_M2", "M2 gate", C[1]), ("flag_M3", "M3 gate", C[0])]:
            ax[1, j].plot(x, 100 * d[col], marker="o", ms=3, color=c, label=lab, lw=1.5)
        ax[0, j].set_title(k, fontsize=8)
        ax[1, j].set_xlabel(xl[k], fontsize=7)
        if k == "occlusion":
            for a in ax[:, j]:
                a.set_xticks(x)
                a.set_xticklabels(d.level, fontsize=6.5)
        if k == "downscale":
            for a in ax[:, j]:
                a.invert_xaxis()
    for j in range(5):
        ax[1, j].set_ylim(0, 105)
        if j:
            ax[1, j].set_yticklabels([])
    ax[0, 0].set_ylabel("median score drift\n(SD units)", fontsize=7)
    ax[1, 0].set_ylabel("% flagged\n(caution/reject)", fontsize=7)
    h1, l1 = ax[0, 2].get_legend_handles_labels()
    h2, l2 = ax[1, 0].get_legend_handles_labels()
    fig.tight_layout(pad=0.3, rect=(0, 0, 1, 0.92))
    fig.legend(h1 + h2, ["drift: " + t for t in l1] + ["flagged: " + t for t in l2],
               loc="upper center", ncol=6, fontsize=6.2, bbox_to_anchor=(0.5, 1.0))
    save(fig, "robustness")


# ---------------------------------------------------------------------- Fig: gate examples
def fig_gate_examples(names_by_status):
    fig, ax = plt.subplots(1, len(names_by_status), figsize=(3.4, 1.55))
    for a, (st, name) in zip(ax, names_by_status):
        img, _ = load_test_image(name)
        m = measure_from_raw(cache[name])
        v = variant_outputs(m)["M3"]
        P = m["mirror"]["lm68_avg"]
        im = viz.draw_axis(viz.draw_mesh(canvas(img), P, DARK, radius=3), m["mirror"]["avg"], P)
        im = viz.status_banner(im, v["status"])
        a.imshow(rgb(im))
        why = ", ".join(sorted({{"abs_yaw": "|yaw|", "abs_pitch": "|pitch|"}.get(r["signal"], r["signal"].replace("_", " ")) for r in
      v["reasons"]
                               if r["level"] == v["status"]})) or "all checks pass"
        a.set_title(f"A = {100*v['score']:.1f}% IOD\n{why}", fontsize=6)
        a.axis("off")
    fig.tight_layout(pad=0.2)
    save(fig, "gate_examples")


if __name__ == "__main__":
    os.makedirs(FIGS, exist_ok=True)
    # Example selection rule (reproducible): the accepted image with median score for the
    # pipeline figure; for the gate figure the median-yaw image of each M3 decision class.
    acc = ok[ok.M3_status == "accept"].sort_values("M3_score")
    ex = acc.iloc[len(acc) // 2]["image"]
    fig_pipeline(ex)
    fig_correspondence(ex)
    fig_agreement()
    fig_pose()
    fig_risk_coverage()
    fig_robustness()
    picks = []
    for st in ("accept", "caution", "reject"):
        s = ok[ok.M3_status == st].sort_values("sig_abs_yaw")
        picks.append((st, s.iloc[len(s) // 2]["image"]))
    fig_gate_examples(picks)
    print("examples:", ex, picks)
```

experiments/make_tables.py

```
"""
Write the paper's LaTeX tables directly from the result CSVs (no hand-typed numbers).
    python experiments/make_tables.py      ->  paper/tables/*.tex  and paper/tables/numbers.tex
```

```
`numbers.tex` defines \\newcommand macros for every number quoted in the text.
"""
import json
import os

import numpy as np
import pandas as pd

from common import TABLES
from fsm.mesh import CONFIG_DIR, ROOT

OUT = os.path.join(ROOT, "paper", "tables")
os.makedirs(OUT, exist_ok=True)
T = lambda n: pd.read_csv(os.path.join(TABLES, n))  # noqa: E731
f2 = lambda x: "--" if pd.isna(x) else f"{x:.2f}"  # noqa: E731
f3 = lambda x: "--" if pd.isna(x) else f"{x:.3f}"  # noqa: E731
pc = lambda x: "--" if pd.isna(x) else f"{100 * x:.1f}"  # noqa: E731
macros = {}


def write(name, body):
    with open(os.path.join(OUT, name), "w") as f:
        f.write(body)


# ---------------------------------------------------------------- agreement (Tab. 3)
ag = T("agreement_global.csv")
cr = T("correspondence_ablation.csv")
mp = T("mapping_nme.csv")
rows = [
    ("M0 baseline~\\cite{nashokan_facialasymmetry} (KD-tree, vert.\\ midline)", ag.iloc[0], "face width"),
    ("\\quad M0 w/o alignment", ag.iloc[1], "face width"),
    ("\\quad M0 + fixed pairs (no KD-tree)", cr.iloc[1], "face width"),
    ("M1: 68-pt pairs + fitted axis", ag.iloc[2], "IOD"),
    ("M3: M1 + mirror averaging", ag.iloc[3], "IOD"),
]
body = ("\\begin{tabular}{@{}lccc@{}}\n\\toprule\nMeasure ($n$=224) & $\\rho$ & $r$ & MAE\\\\\n\\midrule\n")
for name, r, _ in rows:
    body += f"{name} & {f3(r['spearman'])} & {f3(r['pearson'])} & {f2(r['MAE'])}\\\\\n"
body += "\\bottomrule\n\\end{tabular}\n"
write("agreement.tex", body)
macros.update(rhoMzero=f3(ag.iloc[0].spearman), rhoMzeroNoAlign=f3(ag.iloc[1].spearman),
              rhoMzeroFixed=f3(cr.iloc[1].spearman), rhoMone=f3(ag.iloc[2].spearman),
              rhoMthree=f3(ag.iloc[3].spearman), maeMone=f2(ag.iloc[2].MAE),
              maeMthree=f2(ag.iloc[3].MAE),
              kdWrong=f"{100 * cr.iloc[0].kdtree_wrong_partner_rate_mean:.1f}")
g = mp.set_index(["mapping", "points"])
macros.update(nmeFitInner=f2(g.loc[("fitted (train)", "inner51"), "test_NME_mean_pct"]),
              nmeHandInner=f2(g.loc[("hand-made", "inner51"), "test_NME_mean_pct"]),
              nmeFitJaw=f2(g.loc[("fitted (train)", "jaw17"), "test_NME_mean_pct"]),
              nmeHandJaw=f2(g.loc[("hand-made", "jaw17"), "test_NME_mean_pct"]),
              nmeFitInnerMed=f2(g.loc[("fitted (train)", "inner51"), "test_NME_median_pct"]))

# ---------------------------------------------------------------- regions + shrinkage
rg = T("agreement_regions.csv")
sh = T("shrinkage.csv").set_index("region")
body = ("\\begin{tabular}{@{}lcccccc@{}}\n\\toprule\n & \\multicolumn{2}{c}{Spearman $\\rho$} & "
        "\\multicolumn{2}{c}{MAE (\\% IOD)} & \\multicolumn{2}{c}{mean A (\\% IOD)}\\\\\n"
        "\\cmidrule(lr){2-3}\\cmidrule(lr){4-5}\\cmidrule(lr){6-7}\nRegion & M1 & M3 & M1 & M3 & ref & M3\\\\\n\\midrule\n")
for k in ["Brows", "Eyes", "Nose", "Mouth", "Jaw"]:
    a = rg[(rg.region == k) & (rg.variant == "M1")].iloc[0]
    b = rg[(rg.region == k) & (rg.variant == "M3")].iloc[0]
    star = "" if sh.loc[k, "wilcoxon_p"] < 0.01 else "$^{\\dagger}$"
    body += (f"{k} & {f3(a.spearman)} & {f3(b.spearman)} & {f2(a.MAE)} & {f2(b.MAE)} & "
             f"{f2(sh.loc[k, 'ref_mean'])} & {f2(sh.loc[k, 'detector_mean'])}{star}\\\\\n")
body += (f"\\midrule\nGlobal (inner 51) & -- & -- & -- & -- & {f2(sh.loc['global', 'ref_mean'])} & "
         f"{f2(sh.loc['global', 'detector_mean'])}\\\\\n\\bottomrule\n\\end{{tabular}}\n")
write("regions.tex", body)
macros.update(shrinkGlobal=f"{100 * (1 - sh.loc['global', 'ratio']):.1f}",
              shrinkMouth=f"{100 * (1 - sh.loc['Mouth', 'ratio']):.1f}",
              shrinkBrows=f"{100 * (1 - sh.loc['Brows', 'ratio']):.1f}",
              pBrows=f"{sh.loc['Brows', 'wilcoxon_p']:.2f}")

# ---------------------------------------------------------------- ablation (Tab. 4)
ab = T("ablation.csv")
settings = ["clean test (224)", "perturbed: all (pooled)", "perturbed: blur",
            "perturbed: downscale", "perturbed: brightness", "perturbed: occlusion",
            "perturbed: rotation"]
nice = {"clean test (224)": "Clean test", "perturbed: all (pooled)": "Perturbed, pooled",
        "perturbed: blur": "\\quad blur", "perturbed: downscale": "\\quad downscale",
        "perturbed: brightness": "\\quad brightness", "perturbed: occlusion": "\\quad occlusion",
        "perturbed: rotation": "\\quad rotation"}
body = ("\\begin{tabular}{@{}llccccc@{}}\n\\toprule\nSetting & Var. & n & Reported & Accepted & "
        "$\\rho$ & Silent fail. & Drift\\\\\n & & & (\\%) & (\\%) & vs ref & (\\%) $\\downarrow$ & (SD) $\\downarrow$\\\\\n\\midrule\
    n")
for s in settings:
    d = ab[ab.setting == s]
    for i, (_, r) in enumerate(d.iterrows()):
        lab = nice[s] if i == 0 else ""
        n = str(int(r.n)) if i == 0 else ""
        drift = f3(r.get("median_drift_sd", np.nan)) if "median_drift_sd" in r else "--"
```

```
        if r.variant == "M2" and "median_drift_sd" in r and not pd.isna(r.median_drift_sd):
            drift = "="  # same score as M1 (only the gate differs)
        body += (f"{lab} & {r.variant} & {n} & {pc(r.coverage_reported)} & {pc(r.coverage_accepted)} & "
                 f"{f3(r.spearman_vs_ref)} & {pc(r.silent_failure_rate)} & {drift}\\\\\n")
    body += "\\midrule\n" if s != settings[-1] else ""
body += "\\bottomrule\n\\end{tabular}\n"
write("ablation.tex", body)
A = ab.set_index(["setting", "variant"])
for s, tag in [("clean test (224)", "Clean"), ("perturbed: all (pooled)", "Pool"),
               ("perturbed: blur", "Blur"), ("perturbed: downscale", "Down")]:
    for v in ("M1", "M2", "M3"):
        macros[f"silent{tag}{v.replace('M1','One').replace('M2','Two').replace('M3','Three')}"] = pc(A.loc[(s, v), "
     silent_failure_rate"])
    macros[f"cov{tag}Three"] = pc(A.loc[(s, "M3"), "coverage_reported"])
    macros[f"acc{tag}Three"] = pc(A.loc[(s, "M3"), "coverage_accepted"])
macros["driftPoolOne"] = f3(A.loc[("perturbed: all (pooled)", "M1"), "median_drift_sd"])
macros["driftPoolThree"] = f3(A.loc[("perturbed: all (pooled)", "M3"), "median_drift_sd"])
macros["driftPoolZero"] = f3(A.loc[("perturbed: all (pooled)", "M0"), "median_drift_sd"])
macros["driftReduction"] = f"{100 * (1 - A.loc[('perturbed: all (pooled)', 'M3'), 'median_drift_sd'] / A.loc[('perturbed: all (pooled
    )', 'M1'), 'median_drift_sd']):.0f}"

ps = T("perturbation_summary.csv")
rot = ps[ps.perturbation == "rotation"]
macros["rotNoAlignMax"] = f2(rot.drift_M0noalign_sd.max())
macros["rotMthreeMax"] = f3(rot.drift_M3_sd.max())
bl8 = ps[(ps.perturbation == "blur") & (ps.level == "8")].iloc[0]
macros["detBlurEight"] = pc(bl8.detect_rate)
macros["detMin"] = pc(ps.detect_rate.min())

# ---------------------------------------------------------------- pose (Tab. 5)
st = T("pose_strata.csv")
body = ("\\begin{tabular}{@{}lccccc@{}}\n\\toprule\n$|$yaw$|$ (deg) & n & A$_{ref}$ & M1 & "
        "$|$err$|$ M3 & NME\\\\\n\\midrule\n")
for _, r in st.iterrows():
    body += (f"{r.yaw_bin.replace('>=', '$\\geq$')} & {int(r.n)} & {f2(r.A_ref_mean)} & {f2(r.M1_mean)} & "
             f"{f2(r.err_M3)} & {f2(r.nme_inner)}\\\\\n")
body += "\\bottomrule\n\\end{tabular}\n"
write("pose.tex", body)
pcf = T("pose_confound.csv").set_index("subset")
macros.update(rhoYawRef=f2(pcf.loc["all", "rho(A_ref,|yaw|)"]),
              rhoYawRefAcc=f2(pcf.loc["M3 accept", "rho(A_ref,|yaw|)"]),
              rhoYawMthree=f2(pcf.loc["all", "rho(M3,|yaw|)"]),
              yawLow=f2(st.iloc[0].A_ref_mean), yawHigh=f2(st.iloc[-1].A_ref_mean))

# ---------------------------------------------------------------- risk coverage + gate
nm = T("risk_coverage_nme.csv").set_index("signal")
rs = T("risk_coverage_summary.csv").set_index("signal")
op = T("gate_operating_point.csv")
o3 = op[op.variant == "M3"].set_index("status")
macros.update(
    rhoMirrorNme=f2(nm.loc["mirror inconsistency", "spearman_vs_nme"]),
    aurcRandom=f2(nm.loc["random", "AURC_nme"]), aurcMirror=f2(nm.loc["mirror inconsistency", "AURC_nme"]),
    aurcGateTwo=f2(nm.loc["M2 gate risk", "AURC_nme"]), aurcOracle=f2(nm.loc["oracle (true NME)", "AURC_nme"]),
    gapClosed=f"{100 * (nm.loc['random', 'AURC_nme'] - nm.loc['mirror inconsistency', 'AURC_nme']) / (nm.loc['random', 'AURC_nme'] -
     nm.loc['oracle (true NME)', 'AURC_nme']):.0f}",
    aurcAsymRandom=f3(rs.loc["random", "AURC"]),
    aurcAsymMirror=f3(rs.loc["mirror inconsistency only", "AURC"]),
    aurcAsymGate=f3(rs.loc["M2 gate risk (size/blur/exposure/pose)", "AURC"]),
    rhoMirrorErr=f2(rs.loc["mirror inconsistency only", "spearman_risk_vs_err"]),
    pAccept=f"{o3.loc['accept', 'pct']:.1f}", pCaution=f"{o3.loc['caution', 'pct']:.1f}",
    pReject=f"{o3.loc['reject', 'pct']:.1f}",
    nmeAccept=f2(o3.loc["accept", "mean_nme_inner"]), nmeCaution=f2(o3.loc["caution", "mean_nme_inner"]),
)
reasons = T("gate_reasons.csv")
body = "\\begin{tabular}{@{}lcc@{}}\n\\toprule\nSignal & caution & reject\\\\\n\\midrule\n"
nice_sig = {"abs_yaw": "$|$yaw$|$", "abs_pitch": "$|$pitch$|$", "iod_px": "face size (IOD)",
            "mirror_incons": "mirror inconsistency $\\delta$", "brightness_hi": "over-exposure",
            "brightness_lo": "under-exposure", "sharpness": "sharpness", "n_faces": "several faces",
            "clipped": "clipped pixels"}
for sig in reasons.signal.unique():
    c = reasons[(reasons.signal == sig) & (reasons.level == "caution")].n_images.sum()
    r = reasons[(reasons.signal == sig) & (reasons.level == "reject")].n_images.sum()
    body += f"{nice_sig.get(sig, sig)} & {c} & {r}\\\\\n"
body += "\\bottomrule\n\\end{tabular}\n"
write("reasons.tex", body)

rv = T("region_validity_occlusion.csv").set_index("occluded")
macros.update(occEye=pc(rv.loc["one eye", "occluded_region_flagged"]),
              occNose=pc(rv.loc["nose", "occluded_region_flagged"]),
              occNoseOther=pc(rv.loc["nose", "other_regions_flagged"]),
              occMouth=pc(rv.loc["mouth", "occluded_region_flagged"]),
              occMouthOther=pc(rv.loc["mouth", "other_regions_flagged"]))

# ---------------------------------------------------------------- gate thresholds
cfg = json.load(open(os.path.join(CONFIG_DIR, "gate_thresholds.json")))
th = cfg["thresholds"]
body = "\\begin{tabular}{@{}llll@{}}\n\\toprule\nSignal & caution & reject & source\\\\\n\\midrule\n"
src = {"iod_px": "a priori", "abs_yaw": "a priori", "abs_pitch": "a priori"}
units = {"iod_px": " px", "abs_yaw": "$^\\circ$", "abs_pitch": "$^\\circ$"}
for k, r in th.items():
    sym = "$<$" if r["direction"] == "min" else "$>$"
```

```
    s_ = src.get(k, "train pct." if k not in ("brightness_lo", "brightness_hi", "clipped") else "train pct. / a priori")
    u = units.get(k, "")
    body += f"{nice_sig.get(k, k)} & {sym} {r['caution']:.3g}{u} & {sym} {r['reject']:.3g}{u} & {s_}\\\\\n"
body += (f"per-region $\\delta_r$ (validity) & \\multicolumn{{2}}{{l}}{{unreliable if $>$ "
         f"{cfg['region_mirror_limit']:.3g}}} & train pct.\\\\\n\\bottomrule\n\\end{{tabular}}\n")
body = body.replace("sharpness &", "sharpness ($\\log_{10}$ var. Lap.) &").replace("over-exposure &", "mean luma (high) &").replace("
     under-exposure &", "mean luma (low) &")
write("thresholds.tex", body)
macros["alphaIndex"] = f"{cfg['index_alpha_per_pct']:.3f}"
macros["trainMedian"] = f2(1 / cfg["index_alpha_per_pct"])

clean = T("test_clean_rows.csv")
macros["runtime"] = f"{1000 * clean.detect_s.mean():.0f}"
macros["nTrainOk"] = str(int(T("train_clean_rows.csv").ok.sum()))

with open(os.path.join(OUT, "numbers.tex"), "w") as f:
    f.write("% Auto-generated by experiments/make_tables.py -- do not edit by hand\n")
    for k, v in macros.items():
        assert k.isalpha(), k
        f.write(f"\\newcommand{{\\{k}}}{{{v}}}\n")
print(f"wrote {len(macros)} macros and tables to {OUT}")
for k, v in macros.items():
    print(f"  {k} = {v}")

# ---------------------------------------------------------------- code appendix
# LaTeX `listings` cannot typeset some Unicode symbols, so ASCII copies of the source files
# are written to paper/code/ for the appendix. The repository files remain the originals.
import glob  # noqa: E402

ASCII = {"[OK]": "[OK]", "[!]": "[!]", "": "", "[X]": "[X]", "sigma": "sigma",
         "...": "...", "+/-": "+/-", "-": "-", "alpha": "alpha", "*": "*",
         " deg": " deg", "rho": "rho"}
code_dir = os.path.join(ROOT, "paper", "code")
srcs = (glob.glob(os.path.join(ROOT, "src", "fsm", "*.py")) + [os.path.join(ROOT, "app.py")]
        + glob.glob(os.path.join(ROOT, "experiments", "*.py"))
        + glob.glob(os.path.join(ROOT, "tests", "*.py")))
for path in srcs:
    rel = os.path.relpath(path, ROOT)
    txt = open(path, encoding="utf-8").read()
    for k, v in ASCII.items():
        txt = txt.replace(k, v)
    assert all(ord(ch) < 128 for ch in txt), f"non-ASCII left in {rel}"
    dst = os.path.join(code_dir, rel)
    os.makedirs(os.path.dirname(dst), exist_ok=True)
    open(dst, "w").write(txt)
print("ASCII code copies written to", code_dir)

# ---------------------------------------------------------------- figures for the paper
# Copy the generated figures next to the LaTeX source so paper/ is self-contained
# (e.g. it can be uploaded to Overleaf as-is).
import shutil  # noqa: E402

fig_dst = os.path.join(ROOT, "paper", "figures")
os.makedirs(fig_dst, exist_ok=True)
for pdf in glob.glob(os.path.join(ROOT, "results", "figures", "*.pdf")):
    shutil.copy(pdf, fig_dst)
print("figures copied to", fig_dst)
```

tests/test_geometry.py

```
"""Unit tests for the symmetry mathematics (no detector needed)."""
import os
import sys

import numpy as np
import pytest

sys.path.insert(0, os.path.join(os.path.dirname(__file__), "..", "src"))
from fsm import geometry as G  # noqa: E402
from fsm import mesh as MS  # noqa: E402
from fsm.quality import gate_decision, risk_score, DEFAULT_GATE  # noqa: E402

PERM = MS.ibug68_permutation()


def symmetric_face(seed=0):
    """A random but perfectly symmetric 68-point configuration about x = 0."""
    rng = np.random.default_rng(seed)
    P = np.zeros((68, 2))
    for a, b in MS.IBUG68_PAIRS:
        x, y = rng.uniform(10, 60), rng.uniform(-80, 80)
        P[a], P[b] = (-x, y), (x, y)
    for k in MS.IBUG68_CENTRAL:
        P[k] = (0.0, rng.uniform(-80, 80))
    return P


def similarity(P, angle_deg, scale, t):
    a = np.radians(angle_deg)
    R = np.array([[np.cos(a), -np.sin(a)], [np.sin(a), np.cos(a)]])
```

```
    return scale * P @ R.T + np.asarray(t)


def score(P):
    return G.asymmetry(P, PERM, MS.IBUG68_REGIONS, MS.IBUG68_INNER,
                       MS.IBUG68_OUTER_EYE).global_score


def test_permutation_is_involution():
    assert np.all(PERM[PERM] == np.arange(68))
    mp = MS.mp_permutation()
    assert np.all(mp[mp] == np.arange(MS.MP_N))
    # 220 bilateral pairs + 28 midline vertices in the canonical model
    assert np.sum(mp[:468] == np.arange(468)) == 28


def test_reflection_is_its_own_inverse():
    P = np.random.default_rng(1).normal(size=(50, 2))
    Q = G.reflect_points(G.reflect_points(P, 0.7, 3.0), 0.7, 3.0)
    assert np.allclose(P, Q)
    assert np.allclose(G.reflect_vertical(P, 2.0)[:, 0], 4.0 - P[:, 0])


def test_symmetric_face_scores_zero():
    assert score(symmetric_face()) < 1e-6


@pytest.mark.parametrize("angle", [-30, -10, 0, 7, 25])
def test_invariance_to_similarity_transform(angle):
    rng = np.random.default_rng(2)
    P = symmetric_face(3) + rng.normal(scale=1.5, size=(68, 2))   # mildly asymmetric
    s0 = score(P)
    s1 = score(similarity(P, angle, 2.3, (40, -15)))
    assert s0 > 1e-3
    assert abs(s0 - s1) < 1e-4 * max(1, s0) + 1e-6


def test_fitted_axis_recovers_tilt():
    P = similarity(symmetric_face(4), 12.0, 1.0, (5, 5))
    r = G.asymmetry(P, PERM, MS.IBUG68_REGIONS, MS.IBUG68_INNER, MS.IBUG68_OUTER_EYE)
    assert abs(abs(G.axis_tilt_deg(r.theta)) - 12.0) < 0.05


def test_known_displacement_gives_known_score():
    """Moving one eye corner by d pixels horizontally changes its pair residual by ~d."""
    P = symmetric_face(5)
    iod = G.iod(P, MS.IBUG68_OUTER_EYE)
    Q = P.copy()
    Q[39, 0] += 2.0                                # inner corner of one eye, outward shift
    r = G.asymmetry(Q, PERM, MS.IBUG68_REGIONS, MS.IBUG68_INNER, MS.IBUG68_OUTER_EYE)
    # the two partners together deviate by ~2 px from mirror symmetry -> a ~= 1 px / IOD each
    assert r.per_point[39] == pytest.approx(r.per_point[42], rel=1e-6)
    assert r.per_point[39] * iod == pytest.approx(1.0, abs=0.15)
    assert r.region_scores["Eyes"] > r.region_scores["Mouth"]


def test_baseline_kdtree_zero_on_symmetric_mesh():
    """With a vertical, perfectly symmetric mesh the KD-tree baseline gives ~0."""
    V = MS.canonical_vertices()[:, :2] * np.array([20.0, -20.0]) + 300
    L = np.vstack([V, np.tile(V[:1], (10, 1))])        # pad iris rows
    assert G.baseline_kdtree_asymmetry(L)["global_score"] < 1e-6


def test_baseline_not_rotation_invariant_but_proposed_is():
    """The motivating property: vertical-midline methods change under roll."""
    V = MS.canonical_vertices()[:, :2] * np.array([20.0, -20.0])
    rng = np.random.default_rng(6)
    L = np.vstack([V, np.tile(V[:1], (10, 1))]) + rng.normal(scale=0.5, size=(478, 2))
    b0 = G.baseline_kdtree_asymmetry(L)["global_score"]
    b1 = G.baseline_kdtree_asymmetry(similarity(L, 10, 1, (0, 0)))["global_score"]
    assert b1 > 1.5 * b0


def test_unflip_roundtrip():
    """Flipping coordinates and re-indexing twice returns the original landmarks."""
    rng = np.random.default_rng(7)
    L = rng.uniform(0, 500, size=(478, 2))
    perm = MS.mp_permutation()
    W = 640
    Lf = L.copy()
    Lf[:, 0] = W - Lf[:, 0]
    Lf = Lf[perm]                                  # what a perfect detector would return
    assert np.allclose(G.unflip_landmarks(Lf, W, perm), L)


def test_mapping_is_mirror_consistent():
    m = MS.mp68_mapping()
    assert np.all(MS.mp_permutation()[m] == m[PERM])


def test_gate_levels_and_risk():
```

```
ok = {"iod_px": 120, "sharpness": 2.5, "brightness": 120, "clipped": 0.0,
      "abs_yaw": 2, "abs_pitch": 3, "mirror_incons": 0.01, "n_faces": 1}
assert gate_decision(ok, DEFAULT_GATE)[0] == "accept"
assert gate_decision({**ok, "abs_yaw": 15}, DEFAULT_GATE)[0] == "caution"
st, why = gate_decision({**ok, "abs_yaw": 40}, DEFAULT_GATE)
assert st == "reject" and why[0]["signal"] == "abs_yaw"
assert gate_decision({**ok, "brightness": 20}, DEFAULT_GATE)[0] == "reject"
assert risk_score(ok, DEFAULT_GATE) < 0
assert risk_score({**ok, "abs_yaw": 20}, DEFAULT_GATE) == pytest.approx(1.0)
```